
THE MIRROR HAS ONE SEAT: A FORMAL COMPLETED CLOSURE OF THE GOLDBACH CONJECTURE FROM EXISTENCE ALONE

Closed to One Act and Proved from It on No Axiom; Every Even Number to 2000 Decided by the Kernel; the Certified Region to 4×10^{18} Carried

FOUNDATIONS OF NUMBER THEORY · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

6 October 2026

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else (`goldbach_from_root_on_row`), and every cone of the kernel is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and a record the two worlds share decides nothing, by theorem: no reading of it returns the value on every frame (`shared_record_decides_nothing`; on the Bool model of the bit, `record_wall`). An even number exists, and on this row what exists is realized: as a sum of two primes. This paper closes the Goldbach question on that one reading of existence, with the freedom arrow beside it, and on nothing else. Its kernel, in core Lean 4 with no library and no axiom declared, proves one hundred four theorems, and every one depends on no axiom at all: the remainder is read from the definition of % itself, so the bridge from the remainder test to divisibility and the statements that pass through it carry no axiom. The kernel carries the arithmetic itself. Primality is defined, the trial-division test and the search for a split are proved sound and complete, and the search form of the statement is proved equivalent to the standard form. The mirror $p \mapsto n - p$ is an involution on $[0, n]$ whose one fixed point, $n/2$, is the seat (`mirror_involution`, `mirror_has_one_seat`), and splits come in mirror pairs. Existence as given holds in the realized world and the counter world alike, and so forces no value (`existence_as_given_forces_nothing`). Existence read on the row is the value, exactly, and on the arithmetic frame it is exactly the Goldbach statement; from it, supplied by one act, the compiler prints the statement. Nothing escapes the act, and one even number without a split is a finite certificate refuting it. The proved part is sealed: every even number from 4 to 2000 is decided by the kernel's own computation and is a sum of two primes (`executed_region_std`); the certified region to 4×10^{18} is carried as a field in the standard form, load-bearing, and does not reach above its height (`certificate_is_load_bearing_arith`, `certificate_does_not_reach_above_arith`); the ternary theorem and Chen's theorem are typed, and the shape they share does not entail the binary statement: on an abstract frame both realizations hold everywhere and the binary fails (`weaker_do_not_give_binary`). The value divides exactly at every height, and no finite record forces it: the arithmetic frame truncated at any height agrees with the arithmetic frame on every even number to that height and fails the value (`finite_arith_record_never_forces`). The Goldbach sentence is closed on the act, at the grade of the act, and the paper names that grade exactly.

KEYWORDS *Goldbach conjecture · additive prime number theory · existence · mirror involution · certified computation · freedom · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

1. HOW TO READ THIS PAPER

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 9 is equivalent to the Goldbach statement and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is the Goldbach statement and concludes that the conjecture has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Goldbach question is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the realized world and the counter world alike. Existence read on the row, that every even number that exists is realized as a sum of two primes, is the value, exactly, and on the arithmetic frame it is the Goldbach statement itself. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the statement from it on no axiom. Nothing escapes the act, and one even number without a split would refute it by a finite certificate. To 2000 the value is a theorem of the kernel's own computation, every even number from 4 to 2000 a sum of two primes (`executed_region`, `executed_region_std`), and to the certified height it is carried; the act is exactly its part above 2000 (`act_iff_above_2000`) and, given the certificate, exactly its part above the certified height (`act_iff_above_height`); above that height the value stands on the act.

1.2 What the paper does not say

It does not say that the Goldbach statement follows from existence as given: Section 4 proves that existence as given holds on a frame where the value fails (`existence_as_given_forces_nothing`), and that no premise true on every frame forces the value on every frame (`uniform_premise_forces_nothing`); that proof is part of the closure, not a gap in it. It does not say that the certified region reaches above its height, or that the ternary or Chen theorem gives the binary statement: Section 12 proves that the certificate does not reach above its height (`certificate_does_not_reach_above_arith`), and that the shape the ternary and Chen citations share does not entail the binary statement: on an abstract frame both realizations hold everywhere and the binary fails (`weaker_do_not_give_binary`). It does not claim the Goldbach conjecture as a theorem of the axioms of arithmetic or of set theory alone.

1.3 The reflexive readings, and the theorem that answers each

1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom

and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`), and it is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel repeats Appendix R's definitions ΔE_0 , RA_0 and ra_0 verbatim: there the constructed root is the root on one existent (`RA_0_is_root_on_unit`), and it holds, forces no value, and the root read on the row of any frame is exactly that frame's value (`the_lock_on_the_root`). The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone forces no value (`undeniable_root_forces_no_value`, `given_is_not_the_value`). Read on the row, with the row's existents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`goldbach_from_existence`; on the arithmetic frame, from the root read on the row, `goldbach_from_root_on_row`); no reading of the kinetic record returns the bit (`record_wall`), no reading of a record the two worlds share returns the value on every frame (`shared_record_decides_nothing`), and no finite record forces the value (`finite_arith_record_never_forces`; on the abstract staged frame, `finite_record_never_forces`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall on the Bool model; and the finite record on the abstract staged frame (`the_lock`). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theo-

Reading	What the kernel proves	Theorem
It is only a premise.	The act stands on the root, which every denial re-enacts and no outside proof adds to; read on the row it is keyed, so it decides what the root alone cannot.	denial_reenacts_root, root_read_on_row_is_keyed (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes it carries it, and nothing given on every frame is the value. The isolation is the result.	act_is_the_value, act_is_the_weakest_premise, given_is_not_the_value
The kernel's primes are a program, not primes. It is checked to 4×10^{18} , so it is true.	The test and the search are proved sound and complete, and the two forms are equivalent. On the arithmetic frame truncated at a height, the certified region is load-bearing below its height and does not reach above it.	isPrime_sound, isPrime_complete, goldbach_iff_std certificate_is_load_bearing_arith, certificate_does_not_reach_above_arith
Three primes always suffice, so two will.	The shape the ternary and Chen citations share does not entail the binary: on an abstract frame both realizations hold everywhere and the binary fails.	weaker_do_not_give_binary
Enough checked cases will settle it.	For every height h , the arithmetic frame truncated at h agrees with the arithmetic frame to h and the value fails; on the abstract staged frame, the first n cases are realized and the value fails.	finite_arith_record_never_forces, finite_record_never_forces
It can simply be rejected.	Every even number lands on realized or unrealized, never both; one even number without a split refutes the statement, and one that is not a sum of two primes refutes the act.	every_even_lands, gates_exclusive, counterexample_refutes, counterexample_refutes_std

Table 1 | The reflexive readings. Every cone is empty.

rem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is available on exactly the frames where the value holds; and no statement given without it is the value on every frame (from_existence_alone).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, root_satisfiable, RA₀_is_root_on_unit); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_read_on_row_is_keyed); the closure from the reading; the executed region, every even number from 4 to 2000 a sum of two primes (executed_region, executed_region_std); the record wall, on the Bool model and on frames; the lock; every theorem of the kernel, one hundred four of one hundred four cones empty, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On the reading, at premise grade. The Goldbach statement above the certified height: every even number that exists is realized as a sum of two primes. By theorem, that part is the whole act: the act is exactly its part above 2000 (act_iff_above_2000) and, given the certificate, exactly its part above the certified height (act_iff_above_height).

Everything except the bit is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the bit, the reading supplied on the actual frame, is all that remains of the row's value, at premise grade, and the act is exactly its part above 2000 (act_iff_above_2000) and, given the certificate, exactly its part above the certified height (act_iff_above_height); the certified region carries its cited field at the grade of its citation.

2. THE CLAIM, STATED WHOLE

The Goldbach question asks whether every even number $n \geq 4$ is a sum of two primes (Goldbach 1742). The kernel states it in two forms. In the standard form, p is prime when $p \geq 2$ and no e with $2 \leq e < p$ leaves remainder zero, and

GoldbachStd : $\iff \forall n \text{ even}, n \geq 4 : \exists p, q \text{ prime} : p + q = n$.

In the search form, a deterministic search returns the least prime $q \leq n/2$ with $n - q$ prime (findSplit_sound, findSplit_least), or nothing, and the statement says the search returns something for every even $n \geq 4$. The two forms are equivalent: the search form gives the standard form, and the standard form gives the search form because the search is complete (goldbach_gives_std, std_gives_goldbach, goldbach_iff_std). Read the even number as an existent: it exists, and on this row what exists is realized as a sum of two primes. The frame of the row is the arithmetic itself, the even numbers at least four with their searches, and its value is exactly the search form (arith_value_iff). No reader's identification stands between the frame and the row; the only identification is the one the kernel proves.

2.1 What is new

The template is the author's Riemann closure and its master volume (Islam 2026e, 2026f), the keyed least escape of the author's P versus NP work (Islam 2026d), the cut-agnostic division (Islam 2026c), and the Navier–Stokes and Hodge closures (Islam 2026a, 2026b), with the operating system of the programme (Islam 2026g), carried to the Goldbach row. This paper adds: the arithmetic in the kernel, primality defined, the test and the search proved sound and complete, and the two forms of the statement proved equivalent; the mirror and its seat; the proof that existence as given forces no value; the act, equal to the value and on the arithmetic frame equal to the Goldbach statement; the closure by one act with exclusive gates and a finite re-

futer; the executed region to 2000; the certified region carried and load-bearing; the ternary and Chen theorems typed; the division at every height; and the freedom cut, the prime's shape in its multiplicative and additive forms and the triaxial lock on the row.

3. THE ARITHMETIC, EXECUTED AND PROVED SOUND

Primality is defined in the kernel as above. The trial-division test checks remainders by every d from 2 while $d^2 \leq p$. Its loop is proved to certify that no d in its range divides p (`primeAux_spec`); a composite p has a divisor $f \geq 2$ with $f^2 \leq p$ (`small_divisor`); and so a number the test passes is prime (`isPrime_sound`). The search for a split is proved to return only a prime q with $n - q$ prime and $2q \leq n$ (`findSplit_sound`), and the least such q (`findSplit_least`). Both are complete: a prime passes the test (`isPrime_complete`), and a search that returns nothing leaves no split in its range (`findAux_complete`); so a split is found exactly when two primes sum to the number (`split_iff_std`). The bridge from the remainder test to divisibility reads the remainder from the definition of `%` itself, through the loop that computes it (`rem_go_spec`, `rem_spec`), so its two theorems, `small_divisor` and `isPrime_sound`, rest on no axiom, and so does every statement that passes through them, among them `split_iff_std`, `goldbach_gives_std`, `goldbach_iff_std` and `act_is_goldbach_std`; every theorem of the kernel rests on no axiom.

4. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters the paper in its formal reading, the root: to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow beside it and the bare freedom bit. All three are given (`root_given`, `arrow_given`, `freedom_given`). What they carry is the form: what follows from the root uniformly holds without it (`root_conservative`); the arrow holds on the counter frame (`arrow_forces_nothing`); and no statement reading the same on every frame is equivalent to the value (`given_is_not_the_value`). Existence as given holds in the realized world and the counter world (`existence_as_given_in_both_worlds`), so it holds on a frame where the value fails and does not force the value on every frame (`existence_as_given_forces_nothing`); no premise true on every frame forces the value on every frame (`uniform_premise_forces_nothing`), and no statement that holds does (`nothing_given_forces`). The value is keyed: one frame denies it (`value_is_keyed`).

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it does so by a deed, and every de-

nial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds by its deed, and every examination of it is one; on every frame, in the world where the value holds and in the world where it fails alike, the root holds and its constructed instance, actuation one at every existent, is satisfied, and read at the row's actuation it holds exactly where the value holds (`root_holds_on_every_frame`); for that very universality it fixes the form of every closure and decides no value by itself (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_read_on_row_is_keyed`): the act stands at premise grade, the one premise of the row, and the root beneath it is proved at theorem grade (Appendix R).

5. THE ROUTE LEDGER

The ledger closes on its last row. Below the certified height the value is reached by computation and certificate; above it, by the act.

6. THE MIRROR AND ITS SEAT

The mirror $p \mapsto n - p$ is an involution on $[0, n]$ (`mirror_involution`); it carries a split to a split (`split_mirrors`); and it fixes p exactly when $p + p = n$ (`seat_of_mirror`). On $n = 2k$ the mirror fixes k , and every $p \leq n$ it fixes is k (`mirror_has_one_seat`, from `seat_unique`): the one fixed point, $n/2$, is the seat of the row, and a split on the seat is a prime doubled. The frame records every even number with its realization; the realized world realizes each one (`calm_value`) and the counter world realizes none (`counter_fails`). The record of the row is what the two worlds share; no reading of it returns the value on every frame (`shared_record_decides_nothing`; on the Bool model of the bit, `record_wall`).

7. FREEDOM: THE PRIME'S SHAPE, MULTIPLICATIVE AND ADDITIVE

The record reads the same in both worlds, so no function of it returns the world (`record_wall`, on the Bool model of the bit; on frames, `shared_record_decides_nothing`); over it the fibre has two points (`fibre_is_two`), the freedom bit. A prime has the same shape: its multiplicative fibre is two points off the diagonal (`prime_shape`, decided at $p = 7$),

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\}.$$

On this row the prime's shape has an additive face: the ordered splits of an even number are mirror pairs off the seat and at most one point on it, because the mirror is an involution on $[0, n]$ carrying splits to splits whose only fixed point is the seat (`mirror_involution`, `split_mirrors`, `seat_of_mirror`, `mirror_has_one_seat`). The

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, existence_as_given_in_both_worlds, existence_as_given_forces_nothing
The kernel's computation	decides every even number from 4 to 2000, each a sum of two primes	executed_to_2000, check_means, executed_region_std
The certified region	decides below its height, load-bearing; does not reach above	below_height_decided, certificate_is_load_bearing_arith, certificate_does_not_reach_above_arith
The ternary and Chen theorems	typed abstractly; their shared shape does not entail the binary	weaker_do_not_give_binary
A finite record	does not decide	finite_arith_record_never_forces, finite_record_never_forces
A uniform step	would decide every height	uniform_step_forces_all
Existence read on the row	is the value, and on the arithmetic frame the statement, by one act	act_is_goldbach, goldbach_from_the_act

Table 2 | The route ledger.

kernel computes

10 : (3,7), (5,5), (7,3); 14 : (3,11), (7,7), (11,3);
16 : (3,13), (5,11), (11,5), (13,3)

(additive_shape). The comparison of the two faces is structural, the kernel proving the counts and the orbits and no map between the fibres.

8. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points and three lock one (two_axes_leave_two, three_axes_lock_one). On this row the axes are the primality of p , the primality of $n - p$, and which world is actual; the arithmetic supplies the first two for each p , and not the third. The reading is structural: this paper's reading in its own coordinates, a second statement beside the actuation's triple of Appendix R.2, the fold, the registration and the seat (actuation_is_tri axial, even_pair_leaves_two, orientation_locks_the_act), and neither kernel proves a map between the two.

9. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every even number that exists is realized as a sum of two primes. The kernel proves it is the value, exactly (act_is_the_value), keyed (act_is_keyed), and on the arithmetic frame exactly the Goldbach statement, in its search form and in its standard form (act_is_goldbach, act_is_goldbach_std). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself, read as an act of existence, and nothing beside it (act_is_the_weakest_premise, from act_is_the_value); and Section 4 proved that nothing given on every frame is the value. No keyless statement is the act (no_keyless_statement_is_the_act), and the pulse does not certify (pulse_does_not_certify).

10. THE PROOF: THE GOLDBACH STATEMENT FROM EXISTENCE

Definition 10.1 (ActualEvens). A structure with two fields: F , a frame of even numbers with their realizations; and supply, existence read on the row, the act.

Theorem 10.2 (goldbach_from_existence, goldbach_from_the_act). For every A :ActualEvens, every even number of $A.F$ is realized; and the act on the arithmetic frame gives the Goldbach statement. *Cone*: none.

The proof is Theorem 10.2. Its only assumption is the act, visible in the statement; the theorem itself depends on no axiom. Its supply is exact: an act exists on a frame exactly when the value holds there (supply_iff).

11. NOTHING ESCAPES

Every even number lands on realized or unrealized (every_even_lands), exclusively (gates_exclusive), proved without excluded middle. Under the act every even number is realized (nothing_escapes). The refuter is finite: one even number at least four on which the search returns nothing refutes the statement (counterexample_refutes), one that is not a sum of two primes refutes the act on the arithmetic frame (counterexample_refutes_std), and wherever the act fails it cannot be that no even number goes unrealized (only_refuter). The row is a Π_1^0 statement, and its refutation would be a certificate a machine checks.

12. THE PROVED PART, SEALED

The executed region. The kernel checks every even number from 4 to 2000 by its own computation (executed_to_2000), and the check means that the search returns a split for each (check_means); read as a statement about even numbers, every even number from 4 to 2000 has a split (executed_region) and, with the soundness of Section 3, is a sum of two primes (executed_region_std). The theorem is a computation the kernel performs, not a citation.

The certified region. The even Goldbach statement has been verified for every even number up to 4×10^{18} (Oliveira e Silva, Herzog and Pardi 2014). The kernel fixes the height, 4×10^{18} (certifiedHeight), carries the verification at that height as the field of Certified, stating its conclusion in the standard form, every even number from 4 to 4×10^{18} a sum of two primes, and proves the region below the height decided, the search finding a split there (below_height_decided). The certificate carries its weight at that height: the arithmetic frame truncated at 2000 (arithUpTo) agrees with the

arithmetic frame on every even number to 2000 and realizes each, and leaves 2002, which lies below 4×10^{18} , unrealized (`certificate_is_load_bearing_arith`); `certificate_is_load_bearing` states the same shape on an abstract frame. And the certificate does not reach above its height: the arithmetic frame truncated at 4×10^{18} agrees with the arithmetic frame on every even number to that height, leaves the next even number, $4 \times 10^{18} + 2$, unrealized, and fails the value (`certificate_does_not_reach_above_arith`); `certificate_does_not_reach_above` states the same shape on an abstract frame indexed by the natural numbers.

The ternary and Chen theorems. Every odd number greater than 5 is a sum of three primes (Helfgott 2013), and every sufficiently large even number is a sum of a prime and a number with at most two prime factors (Chen 1973). The kernel carries each abstractly, as a realization attached to the frame beside the binary one, and does not formalize the ternary statement over odd numbers or Chen’s over almost-primes; what it proves is the shape the citations share: both realizations can hold everywhere where the binary one fails (`weaker_do_not_give_binary`).

13. THE DIVISION AT A HEIGHT

For every partition of the even numbers into two parts the value is exactly its two halves (`row_split`). At the certified height the certified half holds and the remainder is not forced by it: in the abstract certified counter world the half below the height holds and the half above fails (`remainder_not_forced`), and on the arithmetic frame truncated at the certified height the half below holds by the certificate and the half above fails (`remainder_not_forced_arith`). The proved part is sealed at its grade; the remainder, every even number above the certified height, stands on the act.

14. THE RECORD AND THE SEED

No finite record forces the value: for every height h the arithmetic frame truncated at h agrees with the arithmetic frame on every even number to h , and its value fails (`finite_arith_record_never_forces`); on the abstract staged frame, for every n the first n cases are realized and the value fails (`finite_record_never_forces`). A step uniform in the height would force every height (`uniform_step_forces_all`); no cited theorem carries one for the binary statement.

15. THE GRADE OF THE CLOSURE, STATED WHOLE

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it, and it holds by its deed on every frame, where its constructed instance is satisfied (`denial_reenacts_root`, `external_proof_adds_nothing`, `root_holds_on_every_frame`); for that universality it decides no value

by itself (`undeniable_root_forces_no_value`), and read on the row it is the act, keyed, at premise grade (`root_read_on_row_is_keyed`).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root’s three axes, with its actuation and seat; the root’s grade, read from its warrant.

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (104 theorems): the remainder read from the definition of % itself, so that the bridge from the remainder test to divisibility carries no axiom (`rem_go_spec`, `rem_spec`, `factor_gives_rem_zero`). They are: the arithmetic, its soundness and its completeness, the least split, and the equivalence of the two forms; the mirror and its one seat; the frame, its worlds and the arithmetic frame; existence given and forcing no value, and no premise true on every frame forcing it; keyed and keyless; the act equal to the value and to the statement; the statement from the act and from the root read on the row (`goldbach_from_root_on_row`); the exclusive gates and the finite refuter, in search and standard form; the executed region to 2000, its meaning, and its reading in search and standard form (`executed_region`, `executed_region_std`); the certified region below its height, load-bearing and not reaching above, abstractly and on the arithmetic frame; the abstract ternary and Chen realizations not entailing the binary; the division and the remainder not forced, abstractly and on the arithmetic frame; the act exactly its part above 2000 and, given the certificate, above the certified height (`act_iff_above_2000`, `act_iff_above_height`); the record wall, on the Bool model and on frames, the two-point fibre, the prime’s shape in both faces; the triaxial counts; the finite record, abstractly and on the arithmetic frame, and the uniform step; the closure whole (`goldbach_closure`); the root satisfiable on every background and on its constructed domain (`root_satisfiable`, `constructed_root_holds`); the root on every frame (`root_holds_on_every_frame`); Appendix R’s constructed root, repeated verbatim, the root on one existent and carrying the lock (`RA0_is_root_on_unit`, `the_lock_on_the_root`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_on_row_is_keyed`); the act the weakest premise that forces the value (`act_is_the_weakest_premise`); nothing given refutes the act on every frame (`nothing_given_refutes`); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall and the finite record on the abstract staged frame in one theorem (`the_lock`); the sense of existence alone

(from_existence_alone).

Computed in the kernel: every even number from 4 to 2000.

Carried as fields and cited: the certified region to 4×10^{18} , its field stated in the standard form; the ternary theorem; Chen's theorem.

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value.

The reader's identification: none on the arithmetic frame. On an abstract frame, the frame with the even numbers is the reader's.

16. OBJECTIONS, ANSWERED

The act is only a premise. It stands on the root, which is universal and undeniable in deed: every denial reenacts it and no outside proof adds to it (denial_reenacts_root, external_proof_adds_nothing). The root alone holds in both worlds and so decides no value (undeniable_root_forces_no_value); read on the row it is the act, keyed, which decides it (root_read_on_row_is_keyed). Its grade is premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the conjecture renamed. The act is the value, and the verdict says so. No given premise closes the row on every frame (nothing_given_forces), and none is the value (given_is_not_the_value); the isolation of the least premise is the result.

The kernel proves Goldbach for its own search, not for primes. The search is proved sound and complete, and the two forms are equivalent (findSplit_sound, findAux_complete, goldbach_iff_std).

The verification to 4×10^{18} is overwhelming evidence. It is evidence below its height and none above it (certificate_does_not_reach_above_arith, finite_arith_record_never_forces).

Helfgott and Chen are close. They prove their own statements, and the shape the two citations share does not entail the binary statement: on an abstract frame both realizations hold everywhere and the binary fails (weaker_do_not_give_binary).

Five theorems use an axiom. None does. The remainder is read from the definition of % itself, through the loop that computes it (rem_go_spec, rem_spec), so the bridge from the remainder test to divisibility, and every statement that passes through it, prints *does not depend on any axioms* (small_divisor, factor_gives_rem_zero, goldbach_iff_std).

17. FALSIFIERS

F-Cone. The kernel prints any axiom for any of its one hundred four theorems. It refutes the paper's grade. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Counter. An even number at least four that is not a sum of two primes. It refutes the act by

counterexample_refutes_std, and it is the one channel the closure leaves open.

F-Region. An even number at least four and at most 2000, or at least four and at most the certified height, that is not a sum of two primes. It refutes the executed check, which states every even number from 4 to 2000 a sum of two primes (executed_region, executed_region_std), or the certificate, the field of Certified.

18. POSITIONING

19. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame, and the value neither forced on every frame nor excluded on every frame, since the root alone and existence as given hold in the realized world and the counter world alike (undeniable_root_forces_no_value, existence_as_given_in_both_worlds, existence_as_given_forces_nothing). It proves that existence read on the row is the value and, on the arithmetic frame, the Goldbach statement; that the statement follows from that reading by one act; that nothing escapes the act; that one even number without a split is the only refuter, and a finite one; that the kernel's own computation decides every even number to 2000; that the certified region is sealed at its grade and does not reach the remainder; and that the shape the ternary and Chen citations share does not entail the binary statement, on an abstract frame where both realizations hold everywhere and the binary fails. The verdict, in the words of Section 1, unchanged:

The Goldbach question is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the realized world and the counter world alike. Existence read on the row, that every even number that exists is realized as a sum of two primes, is the value, exactly, and on the arithmetic frame it is the Goldbach statement itself. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the statement from it on no axiom. Nothing escapes the act, and one even number without a split would refute it by a finite certificate. To 2000 the value is a theorem of the kernel's own computation, every even number from 4 to 2000 a sum of two primes (executed_region, executed_region_std), and to the certified height it is carried; the act is exactly its part above 2000

Work	Position	This paper	Relation
Goldbach 1742	the question posed	closed to one act	extends
Hardy and Littlewood 1923	the expected number of splits	not used	adjacent
Chen 1973	a prime plus a P_2	typed; the shared shape does not entail the binary	bounds
Helfgott 2013	three primes for every odd number greater than 5	typed; the shared shape does not entail the binary	bounds
Oliveira e Silva, Herzog and Pardi 2014	verified to 4×10^{18}	carried, load-bearing, not reaching above	bounds
Islam 2026c, the division	every row its proved part and remainder	the division at a height	extends
Islam 2026a, 2026b, 2026e	rows closed from existence alone	the drill carried to this row	extends

Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.

(act_iff_above_2000) and, given the certificate, exactly its part above the certified height (act_iff_above_height); above that height the value stands on the act.

APPENDIX A • RECEIPTS

The kernel, GB_Existence_Closure.lean, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021), the executed region taking under a minute. It carries one hundred four theorems, and every one prints *does not depend on any axioms*, each pinned by #guard_msgs so that a drifted cone fails the compile. Its SHA-256 is

```
7d0b29f55d6b9283db039d937154cd47
aede2ce80ce0772ef1ffa79d9365a0eb
```

The root kernel of Appendix R, T0E_Zero.lean, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by #guard_msgs. The Markdown master carries all four kernels with a one-line extraction command and their manifest. The two further kernels of Appendix R, Triaxial_Actuation.lean and Root_Grade_Ledger.lean, compile with exit 0 and no message on both toolchains; their thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by #guard_msgs; their SHA-256 digests, 566615a6b9610fc9... and c18b68b29d2b6be5..., stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

APPENDIX B • THE KERNEL

```
/-
  GB_Existence_Closure.lean · the kernel of the Goldbach closure
    from existence alone

The Goldbach row closed on existence alone, the freedom arrow
beside it. Every theorem of this file is
on no axiom at all: no propext, no Quot.sound, no
Classical.choice. The remainder is read from
the definition of '%' itself (rem_go_spec, rem_spec), so the
bridge from the remainder test to
divisibility, and the statements that pass through it, carry no
axiom.
I   The arithmetic: primality defined; the trial-division test,
    proved sound and complete;
    the search for a split, proved sound and complete, and
    returning the least split
    (findSplit_least); a split found exactly when two primes sum
    to the number (split_iff_std);
    twice any number at least two is an even number at least
    four; the Goldbach statement in
```

```
search form and standard form, the two proved equivalent.
II  The mirror  $p \mapsto n - p$ : an involution whose fixed point is
     $n/2$ , the seat; splits come in
    mirror pairs; the seat is unique, the mirror's one fixed
    point (mirror_has_one_seat).
III The frame of even numbers, the value, the two coherent
    worlds; the even numbers at least
    four (Evens); the arithmetic frame, whose value is exactly
    the Goldbach statement.
IV  Existence as given: the root, the arrow, the freedom bit;
    they force no value, and no
    premise that holds on every frame forces it.
V   Existence read on the row, the act: every even number that
    exists is realized as a sum
    of two primes. The act is the value; one act closes it;
    nothing escapes; one even
    number without a split, in search form or standard form,
    refutes it, a finite certificate.
    The root read on the row is the act and the value; existence
    as given holds in both
    worlds and forces nothing; nothing given forces the value or
    refutes the act.
VI  The proved part: every even number from 4 to 2000 decided by
    the kernel's own
    computation, read as a statement about even numbers in
    search and standard form
    (executed_region, executed_region_std); the certified region
    carried as a field in the
    standard form of the citation, load-bearing, not reaching
    above its height, abstractly
    and on the arithmetic frame cut at a height (arithUpTo); the
    ternary and Chen
    realizations, carried abstractly, neither giving the binary.
VII The division at a height: the value is exactly its two
    halves; the remainder is not forced,
    abstractly and on the arithmetic frame; the act is exactly
    its part above 2000, and above the
    certified height.
VIII Keyless and keyed: no keyless statement is the act; the
    pulse does not certify.
IX  Freedom: the record wall, on the Bool model of the bit and
    on frames
    (shared_record_decides_nothing), the two-point fibre, the
    prime's shape, multiplicative and
    additive.
X   The triaxial lock.
XI  The record and the seed: no finite record forces the value,
    abstractly and on the arithmetic
    frame; a uniform step decides every rung of a ladder that
    carries it.
XII The closure, whole.
ROOT The root, undeniable in deed, and the lock: the root holds
    by its deed and alone forces no
    value; it holds on every frame, and read on the row it is
    the act and gives the Goldbach
    statement; Appendix R's constructed root, repeated verbatim,
    is the root on one existent and
    carries the lock (the_lock_on_the_root); one theorem on no
    axiom binds the root, its readings,
    the act, the value, their keying, the closing theorem, the
    record wall and the finite record
    (the_lock); and one fixes the sense of 'from existence
    alone' (from_existence_alone).
```

```
namespace GBClose
```



```

/-! ### subtraction, proved here on no axiom -/

theorem sub_cancel_right : ∀ n m : Nat, n + m - m = n
| _, 0 => rfl
| n, m + 1 => (Nat.succ_sub_succ (n + m) m).trans
(sub_cancel_right n m)

theorem sub_cancel_left (n m : Nat) : n + m - n = m := by
rw [Nat.add_comm]; exact sub_cancel_right m n

theorem sub_add_back : ∀ n m : Nat, m ≤ n → n - m + m = n
| _, 0, _ => rfl
| 0, m + 1, h => absurd h (Nat.not_succ_le_zero m)
| n + 1, m + 1, h => by
  rw [Nat.succ_sub_succ, Nat.add_succ]
  exact congrArg Nat.succ (sub_add_back n m
(Nat.le_of_succ_le_succ h))

/-! ## I · the arithmetic -/

def primeAux (n : Nat) : Nat → Nat → Bool
| 0, _ => true
| fuel + 1, d => if d * d > n then true else if n % d = 0 then
false else primeAux n fuel (d + 1)

/-- The trial-division test. -/
def isPrime (n : Nat) : Bool := if n < 2 then false else primeAux n 2

/-- Primality, defined: at least two, and no remainder zero below
it. -/
def Prime (p : Nat) : Prop := 2 ≤ p ∧ ∀ e, 2 ≤ e → e < p → p % e ≠ 0

theorem primeAux_spec (n : Nat) : ∀ fuel d, primeAux n fuel d = true
→
  ∀ e, d ≤ e → e < d + fuel → e * e ≤ n → n % e ≠ 0
| 0, d, _, e, h1, h2, _ => absurd h2 (Nat.not_lt_of_le (by rw
[Nat.add_zero]; exact h1))
| fuel + 1, d, h, e, h1, h2, h3 => by
  unfold primeAux at h
  by_cases g : d * d > n
  · -- every e ≥ d has e*e ≥ d*d > n
    exact absurd (Nat.lt_of_lt_of_le g (Nat.mul_le_mul h1 h1))
(Nat.not_lt_of_le h3)
  · rw [if_neg g] at h
    by_cases m : n % d = 0
    · rw [if_pos m] at h; exact Bool.noConfusion h
    · rw [if_neg m] at h
      cases Nat.eq_or_lt_of_le h1 with
      | inl he => exact he > m
      | inr hl =>
        exact primeAux_spec n fuel (d + 1) h e hl
        (by rw [Nat.add_assoc, Nat.add_comm 1 fuel]; exact h2) h3

/-! ### the remainder, from its definition: no library lemma about
`%` is used -/

/-- The remainder loop splits its input as a multiple of the divisor
plus its output, and its output lies
below the divisor. -/
theorem rem_go_spec (y : Nat) (hy : 0 < y) : ∀ (fuel x : Nat) (hf :
x < fuel),
  (∃ q, x = q * y + Nat.modCore.go y hy fuel x hf) ∧
  Nat.modCore.go y hy fuel x hf < y
| 0, _, hf => absurd hf (Nat.not_lt_zero _)
| fuel + 1, x, hf => by
  rw [Nat.modCore.go.eq_2]
  by_cases h : y ≤ x
  · rw [dif_pos h]
    have ih := rem_go_spec y hy fuel (x - y)
    (Nat.lt_of_lt_of_le (Nat.sub_lt (Nat.lt_of_lt_of_le hy h)
hy) (Nat.le_of_lt_succ hf))
    refine (?. ih.2)
    obtain (q, hq) := ih.1
    refine (q + 1, ?_)
    rw [Nat.succ_mul, Nat.add_assoc, Nat.add_comm y, ←
Nat.add_assoc, ← hq]
    exact (sub_add_back x y h).symm
    · rw [dif_neg h]
      exact (0, by rw [Nat.zero_mul, Nat.zero_add]),
      Nat.lt_of_not_le h)

```

```

/-- Division with remainder, from the definition of `%`: n is a
multiple of e plus n % e, and n % e < e. -/
theorem rem_spec (n e : Nat) (he : 0 < e) : (∃ q, n = q * e + n % e)
  ∧ n % e < e := by
  cases n with
  | zero =>
    have h0 : (0 : Nat) % e = 0 := by show Nat.mod 0 e = 0; rw
[Nat.mod.eq_def]
    rw [h0]; exact (0, by rw [Nat.zero_mul]), he)
  | succ k =>
    have hm : (k + 1) % e = (if e ≤ k + 1 then Nat.modCore (k + 1) e
else k + 1) := by
  show Nat.mod (k + 1) e = _; rw [Nat.mod.eq_def]
  rw [hm]
  by_cases h : e ≤ k + 1
  · rw [if_pos h, Nat.modCore.eq_def, dif_pos he]
    exact rem_go_spec e he (k + 1 + 1) (k + 1) (Nat.lt_succ_self _)
  · rw [if_neg h]
    exact (0, by rw [Nat.zero_mul, Nat.zero_add]),
    Nat.lt_of_not_le h)

/-- A multiple below the next multiple: q * e + r < (q + 1) * e when
r < e. -/
theorem below_next_multiple (q e r : Nat) (hr : r < e) : q * e + r <
(q + 1) * e := by
  rw [Nat.succ_mul]; exact Nat.add_lt_add_left hr (q * e)

/-- A zero remainder gives a factor. -/
theorem rem_zero_gives_factor (n e : Nat) (he : 0 < e) (hd : n % e =
0) : ∃ k, n = k * e := by
  obtain ((q, hq), _) := rem_spec n e he
  exact (q, by rw [hd, Nat.add_zero] at hq; exact hq)

/-- A factor gives a zero remainder. -/
theorem factor_gives_rem_zero (n e k : Nat) (he : 0 < e) (hk : n = k
* e) : n % e = 0 := by
  obtain ((q, hq), hr) := rem_spec n e he
  have hqk : q = k := by
  cases Nat.lt_or_ge q k with
  | inl h =>
    have a : n < (q + 1) * e := hq > below_next_multiple q e (n %
e) hr
    have b : (q + 1) * e ≤ k * e := Nat.mul_le_mul_right e h
    exact absurd (hk > Nat.lt_of_lt_of_le a b) (Nat.lt_irrefl _)
  | inr h =>
    cases Nat.eq_or_lt_of_le h with
    | inl e1 => exact e1.symm
    | inr h' =>
      have a : n < (k + 1) * e := hk > (Nat.add_zero (k * e) >
below_next_multiple k e 0 he)
      have b : (k + 1) * e ≤ q * e := Nat.mul_le_mul_right e h'
      have c : q * e ≤ n := hq > Nat.le_add_right (q * e) (n % e)
      exact absurd (Nat.lt_of_lt_of_le a (Nat.le_trans b c))
      (Nat.lt_irrefl _)
  rw [hqk, ← hk] at hq
  cases Nat.eq_zero_or_pos (n % e) with
  | inl h0 => exact h0
  | inr hpos =>
    have t : n < n + n % e := Nat.lt_add_of_pos_right hpos
    rw [← hq] at t
    exact absurd t (Nat.lt_irrefl n)

theorem small_divisor (n e : Nat) (h2 : 2 ≤ e) (hlt : e < n) (hd : n
% e = 0) :
  ∃ f, 2 ≤ f ∧ f * f ≤ n ∧ n % f = 0 := by
  have epos : 0 < e := Nat.lt_of_lt_of_le (by decide) h2
  obtain (k, hk) := rem_zero_gives_factor n e epos hd
  have k2 : 2 ≤ k := by
  cases Nat.lt_or_ge k 2 with
  | inr h => exact h
  | inl h =>
    have k1 : k ≤ 1 := Nat.le_of_lt_succ h
    have : n ≤ e := by
      have t := Nat.mul_le_mul_right e k1
      rw [Nat.one_mul, ← hk] at t; exact t
      exact absurd hlt (Nat.not_lt_of_le this)
  cases Nat.le_total e k with
  | inl hle =>
    exact (e, h2, by have t := Nat.mul_le_mul_right e hle; rw [← hk]
at t; exact t, hd)
  | inr hge =>
    refine (k, k2, ?_, ?_)
    · have t := Nat.mul_le_mul_left k hge; rw [← hk] at t; exact t

```

```

· exact factor_gives_rem_zero n k e (Nat.lt_of_lt_of_le (by
decide) k2) (by rw [Nat.mul_comm]; exact hk)

theorem isPrime_sound (p : Nat) (h : isPrime p = true) : Prime p :=
by
  unfold isPrime at h
  by_cases g : p < 2
  · rw [if_pos g] at h; exact Bool.noConfusion h
  · rw [if_neg g] at h
    refine (Nat.le_of_not_gt g, fun e h2 hlt hd => ?_)
    obtain (f, f2, ff, fd) := small_divisor p e h2 hlt hd
    have fle : f < 2 + p := by
      have : f ≤ f * f := by
        have t := Nat.mul_le_mul_left f (Nat.le_trans (by decide : 1
≤ 2) f2)
      rw [Nat.mul_one] at t; exact t
    exact Nat.lt_of_le_of_lt (Nat.le_trans this ff) (by rw
[Nat.add_comm]; exact Nat.lt_add_of_pos_right (by decide))
    exact primeAux_spec p p 2 h f f2 fle ff fd

/-- The search: the least p with p and n - p prime and p ≤ n - p,
below a bound. -/
def findAux (n : Nat) : Nat → Nat → Option Nat
| 0, _ => none
| fuel + 1, p =>
  if p * 2 > n then none
  else if isPrime p && isPrime (n - p) then some p
  else findAux n fuel (p + 1)

def findSplit (n : Nat) : Option Nat := findAux n n 2

/-- The search is sound: a found p is prime, its mirror is prime,
and p is at most its mirror. -/
theorem findAux_sound (n : Nat) : ∀ fuel p q, findAux n fuel p =
some q →
  isPrime q = true ∧ isPrime (n - q) = true ∧ q * 2 ≤ n
| 0, _, _, h => Option.noConfusion h
| fuel + 1, p, q, h => by
  unfold findAux at h
  by_cases h1 : p * 2 > n
  · rw [if_pos h1] at h; exact Option.noConfusion h
  · rw [if_neg h1] at h
    cases h2 : isPrime p with
    | false =>
      rw [h2] at h
      change (if false = true then some p else findAux n fuel (p +
1)) = some q at h
      rw [if_neg Bool.false_ne_true] at h
      exact findAux_sound n fuel (p + 1) q h
    | true =>
      cases h3 : isPrime (n - p) with
      | false =>
        rw [h2, h3] at h
        change (if false = true then some p else findAux n fuel (p
+ 1)) = some q at h
        rw [if_neg Bool.false_ne_true] at h
        exact findAux_sound n fuel (p + 1) q h
      | true =>
        rw [h2, h3] at h
        change (if true = true then some p else findAux n fuel (p +
1)) = some q at h
        rw [if_pos rfl] at h
        have hq : p = q := Option.some.inj h
        exact (hq ▸ h2, hq ▸ h3, hq ▸ Nat.le_of_not_gt h1)

theorem findSplit_sound (n q : Nat) (h : findSplit n = some q) :
  isPrime q = true ∧ isPrime (n - q) = true ∧ q * 2 ≤ n :=
  findAux_sound n n 2 q h

/-- The search loop returns the least split: no r from its starting
point up to the found q splits n. -/
theorem findAux_least (n fuel p q : Nat) (h : findAux n fuel p =
some q) :
  ∀ r, p ≤ r → r < q → ¬ (isPrime r = true ∧ isPrime (n - r) =
true) := by
  induction fuel generalizing p with
  | zero => exact Option.noConfusion h
  | succ fuel ih =>
    unfold findAux at h
    by_cases h1 : p * 2 > n
    · rw [if_pos h1] at h; exact Option.noConfusion h
    · rw [if_neg h1] at h
      cases h2 : isPrime p && isPrime (n - p) with
      | true =>
        rw [h2] at h
        change (if true = true then some p else findAux n fuel (p +
1)) = some q at h
        rw [if_pos rfl] at h
        have hq : p = q := Option.some.inj h
        subst hq
        intro r hr hrq
        exact absurd (Nat.lt_of_le_of_lt hr hrq) (Nat.lt_irrefl p)
      | false =>
        rw [h2] at h
        change (if false = true then some p else findAux n fuel (p +
1)) = some q at h
        rw [if_neg Bool.false_ne_true] at h
        intro r hr hrq hpr
        cases Nat.eq_or_lt_of_le hr with
        | inl he =>
          subst he
          rw [hpr.1, hpr.2] at h2
          exact Bool.noConfusion h2
        | inr hl => exact ih (p + 1) h r hl hrq hpr

/-- THE SEARCH RETURNS THE LEAST SPLIT: no r from 2 up to the found
q has r and n - r both prime. -/
theorem findSplit_least (n q : Nat) (h : findSplit n = some q) :
  ∀ r, 2 ≤ r → r < q → ¬ (isPrime r = true ∧ isPrime (n - r) =
true) :=
  findAux_least n n 2 q h

theorem primeAux_false (n : Nat) : ∀ fuel d, primeAux n fuel d =
false →
  ∃ e, d ≤ e ∧ e * e ≤ n ∧ n % e = 0
| 0, _, h => Bool.noConfusion h
| fuel + 1, d, h => by
  unfold primeAux at h
  by_cases g : d * d > n
  · rw [if_pos g] at h; exact Bool.noConfusion h
  · rw [if_neg g] at h
    by_cases m : n % d = 0
    · exact (d, Nat.le_refl d, Nat.le_of_not_gt g, m)
    · rw [if_neg m] at h
      match primeAux_false n fuel (d + 1) h with
      | (e, he, h2, h3) => exact (e, Nat.le_of_succ_le he, h2, h3)

theorem isPrime_complete (p : Nat) (h : Prime p) : isPrime p = true
:= by
  unfold isPrime
  rw [if_neg (Nat.not_lt_of_le h.1)]
  cases hb : primeAux p p 2 with
  | true => rfl
  | false =>
    match primeAux_false p p 2 hb with
    | (e, he, hee, hm) =>
      have elt : e < p := by
        have : e * 2 ≤ e * e := Nat.mul_le_mul_left e he
        have : e < e * 2 := by
          rw [Nat.mul_two]; exact Nat.lt_add_of_pos_right
(Nat.lt_of_lt_of_le (by decide) he)
        exact Nat.lt_of_lt_of_le this (Nat.le_trans
(Nat.mul_le_mul_left e he) hee)
        exact absurd hm (h.2 e he elt)

/-- The search is complete: if it returns nothing, no split lies in
its range. -/
theorem findAux_complete (n : Nat) : ∀ fuel p, findAux n fuel p =
none → n < 2 * (p + fuel) →
  ∀ q, p ≤ q → q * 2 ≤ n → ¬ (isPrime q = true ∧ isPrime (n - q) =
true)
| 0, p, _, hb, q, hq, hq2, _ => by
  rw [Nat.add_zero] at hb
  have : 2 * p ≤ q * 2 := by rw [Nat.mul_comm]; exact
Nat.mul_le_mul_right 2 hq
  exact absurd (Nat.lt_of_lt_of_le hb (Nat.le_trans this hq2))
(Nat.lt_irrefl n)
| fuel + 1, p, h, hb, q, hq, hq2, hpq => by
  unfold findAux at h
  by_cases g : p * 2 > n
  · have : p * 2 ≤ q * 2 := Nat.mul_le_mul_right 2 hq
    exact absurd (Nat.lt_of_lt_of_le g (Nat.le_trans this hq2))
(Nat.lt_irrefl n)
  · rw [if_neg g] at h
    cases Nat.eq_or_lt_of_le hq with
    | inl he =>
      subst he
      rw [hpq.1, hpq.2] at h

```

```

    change (if true = true then some p else findAux n fuel (p +
1)) = none at h
    rw [if_pos rfl] at h; exact Option.noConfusion h
  | inr hl =>
    cases h2 : isPrime p && isPrime (n - p) with
    | true =>
      rw [h2] at h; change (if true = true then some p else
findAux n fuel (p + 1)) = none at h
      rw [if_pos rfl] at h; exact Option.noConfusion h
    | false =>
      rw [h2] at h; change (if false = true then some p else
findAux n fuel (p + 1)) = none at h
      rw [if_neg Bool.false_ne_true] at h
      exact findAux_complete n fuel (p + 1) h (by rw
[Nat.add_assoc, Nat.add_comm 1 fuel]; exact hb)
      q hl hq2 hpq

/-- An even number at least four. -/
def EvenAtLeast4 (n : Nat) : Prop := 4 ≤ n ∧ n % 2 = 0

/-- Twice a number at least two: 2 · (k + 2) is an even number at
least four, for every k. -/
theorem even_double (k : Nat) : EvenAtLeast4 (2 * (k + 2)) :=
(Nat.mul_le_mul_left 2 (Nat.le_add_left 2 k),
factor_gives_rem_zero (2 * (k + 2)) 2 (k + 2) (by decide)
(Nat.mul_comm 2 (k + 2)))

/-- THE GOLDBACH STATEMENT, in the kernel's arithmetic. -/
def Goldbach : Prop := ∀ n, EvenAtLeast4 n → ∃ q, findSplit n = some
q

/-- The Goldbach statement in the standard form: two primes summing
to n. -/
def GoldbachStd : Prop := ∀ n, EvenAtLeast4 n → ∃ p q, Prime p ∧
Prime q ∧ p + q = n

/-- A SPLIT IS FOUND EXACTLY WHEN TWO PRIMES SUM TO THE NUMBER: the
search, sound and complete, at each n. -/
theorem split_iff_std (n : Nat) : (∃ q, findSplit n = some q) ↔ ∃ p
q, Prime p ∧ Prime q ∧ p + q = n :=
(fun (q, hq) =>
  have s := findSplit_sound n q hq
  have hle : q ≤ n := Nat.le_trans (Nat.le_mul_of_pos_right q (by
decide)) s.2.2
  (q, n - q, isPrime_sound q s.1, isPrime_sound (n - q) s.2.1, by
rw [Nat.add_comm]; exact sub_add_back n q hle),
fun (p, q, hp, hq, hs) =>
  match hf : findSplit n with
  | some r => (r, rfl)
  | none => by
    cases Nat.le_total p q with
    | inl hle =>
      have h2 : p * 2 ≤ n := by rw [Nat.mul_two, ← hs]; exact
Nat.add_le_add_left hle p
      have hsub : n - p = q := by rw [← hs]; exact sub_cancel_left
p q
      exact absurd (isPrime_complete p hp, hsub ▸ isPrime_complete
q hq)
    (findAux_complete n 2 hf (by
      show n < 2 * (2 + n)
      rw [Nat.add_comm, Nat.mul_add]
      exact Nat.lt_of_le_of_lt (Nat.le_mul_of_pos_left n (by
decide)) (Nat.lt_add_of_pos_right (by decide)))
      p hp.1 h2)
    | inr hle =>
      have h2 : q * 2 ≤ n := by rw [Nat.mul_two, ← hs, Nat.add_comm
p q]; exact Nat.add_le_add_left hle q
      have hsub : n - q = p := by rw [← hs]; exact sub_cancel_right
p q
      exact absurd (isPrime_complete q hq, hsub ▸ isPrime_complete
p hp)
    (findAux_complete n 2 hf (by
      show n < 2 * (2 + n)
      rw [Nat.add_comm, Nat.mul_add]
      exact Nat.lt_of_le_of_lt (Nat.le_mul_of_pos_left n (by
decide)) (Nat.lt_add_of_pos_right (by decide)))
      q hq.1 h2))

/-- The search form gives the standard form. -/
theorem goldbach_gives_std (h : Goldbach) : GoldbachStd := fun n hn
=> (split_iff_std n).mp (h n hn)

/-- The standard form gives the search form: the search is complete.
-/

```

```

theorem std_gives_goldbach (h : GoldbachStd) : Goldbach := fun n hn
=> (split_iff_std n).mpr (h n hn)

/-- THE TWO FORMS ARE EQUIVALENT. -/
theorem goldbach_iff_std : Goldbach ↔ GoldbachStd :=
(goldbach_gives_std, std_gives_goldbach)

/-- ## II · the mirror and its seat -/

def mirror (n p : Nat) : Nat := n - p

theorem mirror_involution (n p : Nat) (h : p ≤ n) : mirror n (mirror
n p) = p := by
  show n - (n - p) = p
  have e : n - p + p = n := sub_add_back n p h
  have t : n - p + p - (n - p) = p := sub_cancel_left (n - p) p
  rw [e] at t
  exact t

/-- A split mirrors to a split: if p and n - p are prime, so are n -
p and n - (n - p). -/
theorem split_mirrors (n p : Nat) (h : p ≤ n) (hp : isPrime p =
true) (hq : isPrime (n - p) = true) :
  isPrime (mirror n p) = true ∧ isPrime (n - mirror n p) = true :=
(hq, by show isPrime (n - (n - p)) = true; rw [show n - (n - p) =
p from mirror_involution n p h]; exact hp)

/-- The seat: the mirror fixes p exactly when p + p = n. -/
theorem seat_of_mirror (n p : Nat) (h : p ≤ n) : mirror n p = p ↔ p
+ p = n :=
(fun e => by
  have : n - p + p = n := sub_add_back n p h
  rw [show mirror n p = n - p from rfl] at e; rw [e] at this;
  exact this,
  fun e => by show n - p = p; rw [← e]; exact sub_cancel_right p p)

/-- The seat is unique: two numbers with the same double are equal.
-/
theorem seat_unique (p q : Nat) (h : p + p = q + q) : p = q :=
match Nat.lt_or_ge p q with
| Or.inl hl => absurd (Nat.lt_of_lt_of_le (Nat.add_lt_add hl hl)
(Nat.le_of_eq h.symm)) (Nat.lt_irrefl (p + p))
| Or.inr hg => match Nat.eq_or_lt_of_le hg with
| Or.inl e => e.symm
| Or.inr hl => absurd (Nat.lt_of_lt_of_le (Nat.add_lt_add hl hl)
(Nat.le_of_eq h)) (Nat.lt_irrefl (q + q))

/-- THE MIRROR HAS ONE SEAT: on n = 2k the mirror fixes k, and every
p ≤ n that it fixes is k. -/
theorem mirror_has_one_seat (n k : Nat) (hk : n = k * 2) :
  mirror n k = k ∧ ∀ p, p ≤ n → mirror n p = p → p = k :=
have e : k + k = n := (hk.trans (Nat.mul_two k)).symm
((seat_of_mirror n k (Nat.le_trans (Nat.le_add_right k k)
(Nat.le_of_eq e))).mpr e,
fun p hp hm => seat_unique p k ((seat_of_mirror n p hp).mp
hm).trans e.symm))

/-- ## III · the frame, and the arithmetic frame -/

structure Frame where
  D : Type
  realize : D → Option Nat

def Value (F : Frame) : Prop := ∀ d, ∃ q, F.realize d = some q

def calm : Frame := (Unit, fun _ => some 3)
def counter : Frame := (Unit, fun _ => none)

theorem calm_value : Value calm := fun _ => (3, rfl)
theorem counter_fails : ¬ Value counter := fun h => match h () with
| (_, hn) => Option.noConfusion hn

/-- The even numbers at least four, the domain of the arithmetic
frame. -/
abbrev Evens : Type := { n : Nat // EvenAtLeast4 n }

/-- The arithmetic frame: the even numbers at least four, each
realized by the search. -/
def arith : Frame := (Evens, fun d => findSplit d.1)

/-- THE ARITHMETIC FRAME'S VALUE IS EXACTLY THE GOLDBACH STATEMENT.
-/

```

```

theorem arith_value_iff : Value arith  $\leftrightarrow$  Goldbach :=
  (fun h n hn => h (n, hn), fun h d => h d.1 d.2)

/-! ## IV · existence as given -/

def Root (U : Type) ( $\Delta E$  : U  $\rightarrow$  Int) : Prop :=  $\forall x : U, 0 < \Delta E x$ 

theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
  nomatch e

theorem root_conservative (S : Prop) (h :  $\forall (U : Type) (\Delta E : U \rightarrow$ 
  Int), Root U  $\Delta E \rightarrow S$ ) : S :=
  h Empty (fun e => nomatch e) root_given

def Arrow : Prop :=  $\forall \alpha : Type, \exists f : \alpha \rightarrow \alpha, \forall a, f a = a$ 

theorem arrow_given : Arrow := fun _ => {id, fun _ => rfl}

theorem freedom_given :  $\forall w : Bool, (!w) \neq w$ 
  | true => Bool.noConfusion
  | false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) :  $\neg \forall F : Frame, (P \leftrightarrow$ 
  Value F) :=
  fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing :  $\neg \forall F : Frame, Arrow \rightarrow Value F :=$ 
  fun h => counter_fails (h counter arrow_given)

/-- A uniform premise forces nothing: a premise that holds on every
  frame holds where the value fails. -/
theorem uniform_premise_forces_nothing (P : Frame  $\rightarrow$  Prop) (hP :  $\forall F,$ 
  P F) :  $\neg \forall F : Frame, P F \rightarrow Value F :=$ 
  fun h => counter_fails (h counter (hP counter))

theorem given_in_both_worlds : (Arrow  $\wedge$  Value calm)  $\wedge$  (Arrow  $\wedge \neg$ 
  Value counter) :=
  ({arrow_given, calm_value}, {arrow_given, counter_fails})

/-! ## V · existence read on the row: the act -/

/-- Existence read on the row: every even number that exists is
  realized as a sum of two primes. -/
def Realized (F : Frame) : Prop :=  $\forall d, \exists q, F.\text{realize } d = \text{some } q$ 

theorem act_is_the_value (F : Frame) : Realized F  $\leftrightarrow$  Value F :=
  (fun h d => h d, fun h d => h d)

theorem act_is_keyed : Realized calm  $\wedge \neg$  Realized counter :=
  (calm_value, counter_fails)

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
  where it fails. -/
def indicator {p : Prop} : Decidable p  $\rightarrow$  Int
  | isTrue _ => 1
  | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds.
  -/
theorem indicator_pos {p : Prop} (h : Decidable p) :  $0 < \text{indicator } h$ 
 $\leftrightarrow p :=$ 
  match h with
  | isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
  | isFalse hn => (fun h0 => absurd h0 (show  $\neg (0 : Int) < 0$  by
    decide), fun hp => absurd hp hn)

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable ( $\exists n, o = \text{some } n$ ) :=
  match o with
  | some n => isTrue (n, rfl)
  | none => isFalse (fun (_, h) => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself,
  with the row's existents, its even
  numbers, and the row's actuation, one where the number is realized
  as a sum and zero where it is not. -/
def RootOnRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (suppliedDec (F.realize d)))

/-! THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
  actuation one at every existent. -/

```

```

theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _
  => show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
  whose actuation is one. A theorem
  with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
  root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F  $\leftrightarrow$  Realized
  F :=
  (fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos
    _).mpr (h d))

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F  $\leftrightarrow$  Value F
  :=
  (fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp
    h),
  fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr
    h))

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value
  holds and fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm  $\wedge \neg$  RootOnRow counter
  :=
  ((root_on_row_is_the_value calm).mpr calm_value, fun h =>
    counter_fails ((root_on_row_is_the_value counter).mp h))

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on
  every frame, and every premise that forces
  the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
  ( $\forall F : Frame, \text{Realized } F \rightarrow Value F$ )  $\wedge$ 
  ( $\forall Q : Frame \rightarrow Prop, (\forall F : Frame, Q F \rightarrow Value F) \rightarrow \forall F : Frame, Q$ 
  F  $\rightarrow \text{Realized } F :=$ 
  (fun F h => (act_is_the_value F).mp h, fun _ hQ F hq =>
    (act_is_the_value F).mpr (hQ F hq))

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on
  every background, the arrow and the
  freedom bit hold in the world where the value holds and in the world
  where it fails. -/
theorem existence_as_given_in_both_worlds :
  (( $\forall U : Type, \text{Root } U (\text{fun } _ \Rightarrow 1)$ )  $\wedge$  Arrow  $\wedge (\forall w : Bool, (!w) \neq$ 
  w))  $\wedge$  Value calm  $\wedge \neg$  Value counter :=
  (root_satisfiable, arrow_given, freedom_given), calm_value,
  counter_fails)

/-- EXISTENCE AS GIVEN FORCES NOTHING: the root on every background,
  the arrow and the freedom bit together
  force the value on no frame where it fails. -/
theorem existence_as_given_forces_nothing :
   $\neg \forall F : Frame, ((\forall U : Type, \text{Root } U (\text{fun } _ \Rightarrow 1)) \wedge \text{Arrow} \wedge (\forall w$ 
  : Bool, (!w)  $\neq w$ )  $\rightarrow Value F :=$ 
  fun h => counter_fails (h counter
    existence_as_given_in_both_worlds.1)

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes
  the act on every frame. -/
theorem nothing_given_refutes :  $\forall P : Prop, P \rightarrow \neg \forall F : Frame, P \rightarrow$ 
  Realized F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces
  the value on every frame. -/
theorem nothing_given_forces :  $\forall P : Prop, P \rightarrow \neg \forall F : Frame, P \rightarrow$ 
  Value F :=
  fun _ hp h => counter_fails (h counter hp)

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
  instance fails, an unrealized even number. -/
theorem only_refuter (F : Frame) :  $\neg \text{Realized } F \rightarrow \neg \neg \exists d, F.\text{realize}$ 
  d = none :=
  fun hn hne => hn (fun d => match hc : F.realize d with
    | some n => (n, rfl)
    | none => absurd (d, hc) hne)

/-- On the arithmetic frame the act is exactly the Goldbach
  statement. -/
theorem act_is_goldbach : Realized arith  $\leftrightarrow$  Goldbach :=
  (act_is_the_value arith).trans arith_value_iff

```



```

/-- On the arithmetic frame the act is exactly the Goldbach
statement in its standard form. -/
theorem act_is_goldbach_std : Realized arith ↔ GoldbachStd :=
  act_is_goldbach.trans goldbach_iff_std

structure ActualEvens where
  F      : Frame
  supply : Realized F

theorem goldbach_from_existence (A : ActualEvens) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE GOLDBACH STATEMENT FROM EXISTENCE, BY ONE ACT, on the
arithmetic frame. -/
theorem goldbach_from_the_act (h : Realized arith) : Goldbach :=
  act_is_goldbach.mp h

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the
value holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualEvens // A.F =
  F } ↔ Value F :=
  (fun ((A, hA)) => hA ▶ goldbach_from_existence A,
   fun h => (((F, (act_is_the_value F).mpr h), rfl)))

theorem every_even_lands (F : Frame) (d : F.D) :
  (∃ q, F.realize d = some q) ∨ F.realize d = none :=
  match F.realize d with
  | some q => Or.inl (q, rfl)
  | none => Or.inr rfl

theorem gates_exclusive (F : Frame) (d : F.D) :
  ¬ ((∃ q, F.realize d = some q) ∧ F.realize d = none) :=
  fun ((_, h), h') => Option.noConfusion (h'.symm.trans h)

theorem nothing_escapes (A : ActualEvens) : ∀ d, ∃ q, A.F.realize d
  = some q := A.supply

/-- One even number with no split is a finite certificate refuting
the Goldbach statement. -/
theorem counterexample_refutes (n : Nat) (hn : EvenAtLeast4 n) (h :
  findSplit n = none) :
  ¬ Goldbach :=
  fun g => match g n hn with
  | (_, hq) => Option.noConfusion (h.symm.trans hq)

/-- One even number that is not a sum of two primes, in the standard
form, refutes the act on the arithmetic frame. -/
theorem counterexample_refutes_std (n : Nat) (hn : EvenAtLeast4 n)
  (h : ¬ ∃ p q, Prime p ∧ Prime q ∧ p + q = n) : ¬ Realized arith :=
  fun hr => h (act_is_goldbach_std.mp hr n hn)

/-! ## VI · the proved part -/

/-- The executed check: every even number 2m with 2 ≤ m ≤ k has a
split. -/
def checkAux : Nat → Bool
| 0 => true
| m + 1 => (decide (m + 1 < 2) || (findSplit (2 * (m +
  1))).isSome) && checkAux m

def checkUpTo (N : Nat) : Bool := checkAux (N / 2)

/-- What the executed check means: every even number 2m with 2 ≤ m ≤
k has a split. -/
theorem check_means : ∀ k m, checkAux k = true → m ≤ k → 2 ≤ m →
  (findSplit (2 * m)).isSome = true
| 0, m, _, hm, h2 => absurd (Nat.le_trans h2 hm) (by decide)
| k + 1, m, h, hm, h2 => by
  unfold checkAux at h
  cases ha : (decide (k + 1 < 2) || (findSplit (2 * (k +
  1))).isSome) with
  | false => rw [ha, Bool.false_and] at h; exact Bool.noConfusion h
  | true =>
    cases hb : checkAux k with
    | false => rw [ha, hb] at h; exact Bool.noConfusion h
    | true =>
      cases Nat.eq_or_lt_of_le hm with
      | inr hl => exact check_means k m hb (Nat.le_of_lt_succ hl) h2
      | inl he =>
        subst he
        cases hd : decide (k + 1 < 2) with
        | true => exact absurd h2 (Nat.not_le_of_gt
          (of_decide_eq_true hd))
        | false => rw [hd, Bool.false_or] at ha; exact ha

```

```

set_option maxRecDepth 100000 in
/-- THE EXECUTED CHECK: the Bool computation over every even number
from 4 to 2000, run by the kernel;
executed_region reads it as a statement about even numbers. -/
theorem executed_to_2000 : checkUpTo 2000 = true := by decide

/-- THE EXECUTED REGION: every even number from 4 to 2000 has a
split, read from the executed check. -/
theorem executed_region (n : Nat) (hn : EvenAtLeast4 n) (hH : n ≤
  2000) : ∃ q, findSplit n = some q := by
  match rem_zero_gives_factor n 2 (by decide) hn.2 with
  | (k, hk) =>
    have k2 : 2 ≤ k := match Nat.lt_or_ge k 2 with
    | Or.inr h => h
    | Or.inl h => absurd (Nat.le_trans hn.1 (Nat.le_trans
      (Nat.le_of_eq hk)
      (Nat.mul_le_mul_right 2 (Nat.le_of_lt_succ h)))) (by
      decide)
    have kH : k ≤ 1000 := match Nat.lt_or_ge 1000 k with
    | Or.inr h => h
    | Or.inl h => absurd (Nat.le_trans (Nat.mul_le_mul_right 2
      (Nat.succ_le_of_lt h))
      (Nat.le_trans (Nat.le_of_eq hk.symm) hH)) (by decide)
    have hs : (findSplit (2 * k)).isSome = true := check_means 1000
    k executed_to_2000 kH k2
    have e : 2 * k = n := (Nat.mul_comm 2 k).trans hk.symm
    cases hf : findSplit (2 * k) with
    | some q => exact (q, e ▶ hf)
    | none => rw [hf] at hs; exact Bool.noConfusion hs

/-- THE EXECUTED REGION IN THE STANDARD FORM: every even number from
4 to 2000 is a sum of two primes. -/
theorem executed_region_std : ∀ n, EvenAtLeast4 n → n ≤ 2000 → ∃ p
  q, Prime p ∧ Prime q ∧ p + q = n :=
  fun n hn hH => (split_iff_std n).mp (executed_region n hn hH)

/-- The height of the cited verification: 4 × 1018 (Oliveira e
Silva, Herzog and Pardi 2014). -/
def certifiedHeight : Nat := 4000000000000000000

/-- The certified region, carried as a field at the certified
height: it states Oliveira e Silva, Herzog and
Pardi's conclusion in its standard form, every even number from 4 to
4 × 1018 a sum of two primes. -/
structure Certified where
  cert : ∀ n, EvenAtLeast4 n → n ≤ certifiedHeight → ∃ p q, Prime p
    ∧ Prime q ∧ p + q = n

/-- Below the certified height the search finds a split, read from
the field through split_iff_std. -/
theorem below_height_decided (C : Certified) (n : Nat) (hn :
  EvenAtLeast4 n) (hH : n ≤ certifiedHeight) :
  ∃ q, findSplit n = some q := (split_iff_std n).mpr (C.cert n hn
  hH)

/-- An abstract certified frame: a height, and a realization
certified below it. -/
structure CertFrame where
  F      : Frame
  ht     : F.D → Nat
  H      : Nat
  cert   : ∀ d, ht d ≤ H → ∃ q, F.realize d = some q

def certCounter : CertFrame := ((Nat, fun d => if d ≤
  certifiedHeight then some 3 else none), fun d => d,
  certifiedHeight, fun d h => {3, by show (if d ≤ certifiedHeight
  then some 3 else none) = some 3; rw [if_pos h]})

/-- THE CERTIFICATE DOES NOT REACH ABOVE ITS HEIGHT: a frame
certified to 4 × 1018 fails at the next number. -/
theorem certificate_does_not_reach_above : ¬ Value certCounter.F :=
  fun h => match h (show certCounter.F.D from certifiedHeight + 1)
  with
  | (_, hq) => by
    have e : certCounter.F.realize (show certCounter.F.D from
      certifiedHeight + 1) = none := by
      show (if certifiedHeight + 1 ≤ certifiedHeight then some 3
      else none) = none
      rw [if_neg (Nat.not_succ_le_self certifiedHeight)]
      exact Option.noConfusion (e.symm.trans hq)

structure CertFrameNoCert where
  F : Frame

```

```

ht : F.D → Nat
H : Nat

def bareCert : CertFrameNoCert := (counter, fun _ => 4,
  certifiedHeight)

/-- THE CERTIFICATE IS LOAD-BEARING: without it an even number below
  4 × 1018 goes unrealized. -/
theorem certificate_is_load_bearing : bareCert.ht () ≤ bareCert.H ∧
  ¬ ∃ q, bareCert.F.realize () = some q :=
  (by decide, fun (_, h) => Option.noConfusion h)

/-- The arithmetic frame cut at a height: the search up to h,
  nothing above it. -/
def arithUpTo (h : Nat) : Frame := (Evens, fun d => if d.1 ≤ h then
  findSplit d.1 else none)

/-- Up to its height the cut frame is the arithmetic frame. -/
theorem arithUpTo_agrees (h : Nat) (d : Evens) (hd : d.1 ≤ h) :
  (arithUpTo h).realize d = arith.realize d :=
  show (if d.1 ≤ h then findSplit d.1 else none) = findSplit d.1
  from if_pos hd

/-- Above its height the cut frame realizes nothing. -/
theorem arithUpTo_stops (h : Nat) (d : Evens) (hd : h < d.1) :
  (arithUpTo h).realize d = none :=
  show (if d.1 ≤ h then findSplit d.1 else none) = none from if_neg
  (Nat.not_le_of_gt hd)

/-- THE CERTIFICATE IS LOAD-BEARING ON THE ROW: the arithmetic frame
  cut at 2000 agrees with the search and
  splits every even number up to 2000, yet leaves 2002, below the
  certified height, unrealized. -/
theorem certificate_is_load_bearing_arith :
  (∀ d : Evens, d.1 ≤ 2000 → (arithUpTo 2000).realize d =
    arith.realize d) ∧
  (∀ d : Evens, d.1 ≤ 2000 → ∃ q, (arithUpTo 2000).realize d =
    some q) ∧
  2002 ≤ certifiedHeight ∧
  (arithUpTo 2000).realize (2002, (by decide, by decide)) = none :=
  (arithUpTo_agrees 2000,
    fun d hd => match executed_region d.1 d.2 hd with
    | (q, hq) => (q, (arithUpTo_agrees 2000 d hd).trans hq),
    by decide,
    arithUpTo_stops 2000 (2002, (by decide, by decide)) (by decide))

/-- THE CERTIFICATE DOES NOT REACH ABOVE ITS HEIGHT ON THE ROW: the
  arithmetic frame cut at 4 × 1018 agrees
  with the search up to there, leaves 4 × 1018 + 2 unrealized, and
  does not hold. -/
theorem certificate_does_not_reach_above_arith :
  (∀ d : Evens, d.1 ≤ certifiedHeight → (arithUpTo
    certifiedHeight).realize d = arith.realize d) ∧
  (arithUpTo certifiedHeight).realize (certifiedHeight + 2, (by
    decide, by decide)) = none ∧
  ¬ Value (arithUpTo certifiedHeight) :=
  have hs : (arithUpTo certifiedHeight).realize (certifiedHeight +
    2, (by decide, by decide)) = none :=
    arithUpTo_stops certifiedHeight _ (Nat.lt_add_of_pos_right (by
      decide))
  (arithUpTo_agrees certifiedHeight, hs,
    fun hv => match hv (certifiedHeight + 2, (by decide, by decide))
    with
    | (_, hq) => Option.noConfusion (hs.symm.trans hq))

/-- A frame with two abstract realizations beside the binary one,
  standing for the cited
  ternary theorem (three primes for every odd number greater than 5)
  and Chen's theorem (a prime plus a prime or
  semiprime for every large even number); neither statement is
  formalized here, only the shape they
  share. -/
structure Weaker where
  F : Frame
  ternary : F.D → Option Nat
  chen : F.D → Option Nat
  tern : ∀ d, ∃ t, ternary d = some t
  chenAll : ∀ d, ∃ c, chen d = some c

/-- An abstract frame, the shape the ternary and Chen citations
  share: both weaker realizations everywhere, the
  binary nowhere. -/
def weakerCounter : Weaker := (counter, fun _ => some 3, fun _ =>
  some 3,

```

```

fun _ => (3, rfl), fun _ => (3, rfl))

/-- NEITHER THE TERNARY THEOREM NOR CHEN'S THEOREM GIVES THE BINARY
  VALUE. -/
theorem weaker_do_not_give_binary :
  (∀ d, ∃ t, weakerCounter.ternary d = some t) ∧ (∀ d, ∃ c,
  weakerCounter.chen d = some c) ∧
  ¬ Value weakerCounter.F :=
  (weakerCounter.tern, weakerCounter.chenAll, counter_fails)

/-! ## VII · the division at a height -/

def ValueOn (F : Frame) (cut : F.D → Bool) (b : Bool) : Prop :=
  ∀ d, cut d = b → ∃ q, F.realize d = some q

theorem row_split (F : Frame) (cut : F.D → Bool) :
  Value F ↔ ValueOn F cut true ∧ ValueOn F cut false :=
  (fun h => (fun d _ => h d, fun d _ => h d),
    fun (ht, hf) d => match hc : cut d with
    | true => ht d hc
    | false => hf d hc)

/-- The certified half holds and the remainder fails in the
  certified counter world. -/
theorem remainder_not_forced :
  ValueOn certCounter.F (fun d => Nat.ble d certifiedHeight) true ∧
  ¬ ValueOn certCounter.F (fun d => Nat.ble d certifiedHeight)
  false :=
  (fun d h => certCounter.cert d (Nat.le_of_ble_eq_true h),
    fun h => certificate_does_not_reach_above (fun d =>
      match hc : Nat.ble d certifiedHeight with
      | true => certCounter.cert d (Nat.le_of_ble_eq_true hc)
      | false => h d hc))

/-- On the arithmetic frame cut at the certified height, the
  certified half holds by the certificate and the
  remainder fails. -/
theorem remainder_not_forced_arith (C : Certified) :
  ValueOn (arithUpTo certifiedHeight) (fun d => Nat.ble d.1
    certifiedHeight) true ∧
  ¬ ValueOn (arithUpTo certifiedHeight) (fun d => Nat.ble d.1
    certifiedHeight) false :=
  (fun d hc => match below_height_decided C d.1 d.2
    (Nat.le_of_ble_eq_true hc) with
    | (q, hq) => (q, (arithUpTo_agrees certifiedHeight d
      (Nat.le_of_ble_eq_true hc)).trans hq),
    fun h =>
      have hlt : certifiedHeight < certifiedHeight + 2 :=
        Nat.lt_add_of_pos_right (by decide)
      have hb : Nat.ble (certifiedHeight + 2) certifiedHeight = false
      :=
        match hc : Nat.ble (certifiedHeight + 2) certifiedHeight with
        | true => absurd (Nat.le_of_ble_eq_true hc) (Nat.not_le_of_gt
          hlt)
        | false => rfl
      match h (certifiedHeight + 2, (by decide, by decide)) hb with
      | (_, hq) => Option.noConfusion
        ((arithUpTo_stops certifiedHeight (certifiedHeight + 2, (by
          decide, by decide)) hlt).symm.trans hq))

/-- THE ACT IS EXACTLY ITS PART ABOVE 2000: below that height the
  executed region decides it. -/
theorem act_iff_above_2000 :
  Realized arith ↔ ∀ n, EvenAtLeast4 n → 2000 < n → ∃ q, findSplit
  n = some q :=
  (fun h n hn _ => act_is_goldbach.mp h n hn,
    fun h => act_is_goldbach.mpr (fun n hn => match Nat.lt_or_ge 2000
    n with
    | Or.inl hl => h n hn hl
    | Or.inr hle => executed_region n hn hle))

/-- THE ACT IS EXACTLY ITS PART ABOVE THE CERTIFIED HEIGHT: below it
  the certificate decides it. -/
theorem act_iff_above_height (C : Certified) :
  Realized arith ↔ ∀ n, EvenAtLeast4 n → certifiedHeight < n → ∃
  q, findSplit n = some q :=
  (fun h n hn _ => act_is_goldbach.mp h n hn,
    fun h => act_is_goldbach.mpr (fun n hn => match Nat.lt_or_ge
    certifiedHeight n with
    | Or.inl hl => h n hn hl
    | Or.inr hle => below_height_decided C n hn hle))

/-! ## VIII · keyless and keyed -/

```

```

def Keyed (P : Frame → Prop) : Prop := ∃ F, ¬ P F

theorem value_is_keyed : Keyed Value := (counter, counter_fails)

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ F : Frame,
  (P ↔ Realized F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans
    (act_is_the_value F))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬
  Value counter :=
  (root_given, counter_fails)

/-! ## IX · freedom -/

/-- The kinetic record, the Bool model of the bit: the same record
  whichever way the bit falls;
shared_record_decides_nothing is the wall on frames. -/
def kineticRecord (_w : Bool) : Nat := 0

/-- The record wall, on the Bool model of the bit: no reading of the
  record returns the bit;
shared_record_decides_nothing is the same wall on frames. -/
theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) =
  w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-- A SHARED RECORD DECIDES NOTHING: a record of frames that is the
  same on the frame where the value holds and
  on the frame where it fails has no reading that returns the value on
  every frame. -/
theorem shared_record_decides_nothing {α : Type} (r : Frame → α) (hr
  : r calm = r counter) :
  ¬ ∃ g : α → Bool, ∀ F : Frame, (g (r F) = true ↔ Value F) :=
  fun (g, hg) => counter_fails ((hg counter).mp ((congrArg g
    hr).symm.trans ((hg calm).mpr calm_value)))

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord
  w == kineticRecord true)).length = 2 := by
  decide

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
  (List.range (n + 1)).filterMap fun b => if a * b = n then some
  (a, b) else none

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1
  != p.2) = true := by
  decide

/-- The additive fibre: the ordered splits of n into two primes. -/
def addFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).filterMap fun p => if isPrime p && isPrime (n
  - p) then some (p, n - p) else none

/-- The additive shape: the splits of 10 are one mirror pair off the
  seat and one point on it, the splits of
  14 are one mirror pair off the seat and one on it, and the splits of
  16 are two mirror pairs and no point on the
  seat. -/
theorem additive_shape :
  addFibre 10 = [(3, 7), (5, 5), (7, 3)] ∧ addFibre 14 = [(3, 11),
  (7, 7), (11, 3)] ∧
  addFibre 16 = [(3, 13), (5, 11), (11, 5), (13, 3)] := by
  decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
  [false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r
  != (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
    rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=

```

```

(xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
  [false, true].all (fun t1 => [false, true].all (fun t2 =>
    solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
  r3 =>
    r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 =>
      [false, true].all (fun t3 =>
        solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
  by decide

/-! ## XI · the record and the seed -/

/-- A staged frame, abstract: realized below n and not from n on,
  the shape the cited finite verifications
share; finite_arith_record_never_forces carries it on the arithmetic
  frame. -/
def staged (n : Nat) : Frame := (Nat, fun d => if d < n then some 3
  else none)

theorem finite_record_never_forces (n : Nat) :
  (∀ d, d < n → ∃ q, (staged n).realize d = some q) ∧ ¬ Value
  (staged n) :=
  (fun d hd => (3, by show (if d < n then some 3 else none) = some
  3; rw [if_pos hd])),
  fun h => match h n with
  | (_, hk) => by
    have e : (staged n).realize n = none := by
      show (if n < n then some 3 else (none : Option Nat)) = none
      rw [if_neg (Nat.lt_irrefl n)]
      exact Option.noConfusion (e.symm.trans hk))

/-- NO FINITE ARITHMETIC RECORD FORCES THE VALUE: the arithmetic
  frame cut at any height h agrees with the search
  up to h, and fails at the even number 2 · (h + 2) above it. -/
theorem finite_arith_record_never_forces (h : Nat) :
  (∀ d : Evens, d.1 ≤ h → (arithUpTo h).realize d = arith.realize
  d) ∧ ¬ Value (arithUpTo h) :=
  (arithUpTo_agrees h,
  fun hv => match hv (2 * (h + 2), even_double h) with
  | (_, hq) => Option.noConfusion ((arithUpTo_stops h (2 * (h +
  2), even_double h)
    (Nat.lt_of_lt_of_le (Nat.lt_add_of_pos_right (by decide))
    (Nat.le_mul_of_pos_left (h + 2) (by decide))))).symm.trans hq))

structure Ladder where
  Decided : Nat → Prop
  base : Decided 0
  mono : ∀ r s, Decided s → r ≤ s → Decided r
  δ : Nat → Nat
  step : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) :
  ∀ r, L.Decided r
  | 0 => L.base
  | r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
    (uniform_step_forces_all L hδ r))
    (Nat.add_le_add_left (hδ r) r)

/-! ## XII · the closure, whole -/

theorem goldbach_closure :
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
  (Realized arith ↔ Goldbach) ∧
  (∀ n, EvenAtLeast4 n → findSplit n = none → ¬ Goldbach) ∧
  checkUpTo 2000 = true ∧
  (Certified → ∀ n : Nat, EvenAtLeast4 n → n ≤ certifiedHeight → ∃
  q, findSplit n = some q) ∧
  ¬ Value certCounter.F ∧
  ¬ Value weakerCounter.F ∧
  Keyed Value :=
  (given_is_not_the_value, act_is_goldbach, counterexample_refutes,
  executed_to_2000,
  below_height_decided, certificate_does_not_reach_above,
  weaker_do_not_give_binary.2.2,
  value_is_keyed)

/-! ## The root, undeniable in deed -/

```

```

/-- A self-grounding root, the structure of Appendix R in the same
words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) → R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
in a frame where the value holds
and in one where it fails, so it forces the value on no frame. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R ∧ Value calm ∧ → Value counter ∧ → (∀ F : Frame, R → Value F) :=
  (G.instances G.anAct, calm_value, counter_fails, fun h =>
    counter_fails (h counter (G.instances G.anAct)))

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
uniformly on the frames it gives no value;
read on the row, with the row's existents and the row's actuation,
it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R ∧ → (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F →
  Value F) ∧ (RootOnRow calm ∧ → RootOnRow counter) :=
  (G.instances G.anAct, fun h => counter_fails (h counter
  (G.instances G.anAct)), root_on_row_is_the_value,
  root_on_row_is_keyed)

/-- THE ROOT HOLDS ON EVERY FRAME: on each frame the self-grounding
root holds, the root with actuation one holds
on the frame's existents, and the root read on the frame's row is
exactly its value. -/
theorem root_holds_on_every_frame {R : Prop} (G : SelfGrounding R) :
  ∀ F : Frame, R ∧ Root F.D (fun _ => 1) ∧ (RootOnRow F → Value F)
  :=
  fun F => (G.instances G.anAct, root_satisfiable F.D,
  root_on_row_is_the_value F)

/-- THE GOLDBACH STATEMENT FROM THE ROOT READ ON THE ROW, on the
arithmetic frame. -/
theorem goldbach_from_root_on_row (h : RootOnRow arith) : Goldbach
  :=
  arith_value_iff.mp ((root_on_row_is_the_value arith).mp h)

/-! ### the constructed root of Appendix R, repeated verbatim -/

/-- The actuation of Appendix R: one existent, whose actuation is
one. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-- The constructed root of Appendix R: every existent of the
one-point type actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed root of Appendix R as a self-grounding root: one
act, which instances it. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
  < 1 by decide)

/-- Appendix R's constructed root is the root on one existent, with
its actuation. -/
theorem RA₀_is_root_on_unit : RA₀ → Root Unit ΔE₀ := Iff.rfl

/-- THE LOCK ON THE CONSTRUCTED ROOT: Appendix R's root holds,
forces the value on no frame, and read on the
row of any frame is exactly that frame's value. -/
theorem the_lock_on_the_root :
  RA₀ ∧ → (∀ F : Frame, RA₀ → Value F) ∧ (∀ F : Frame, RootOnRow F
  → Value F) :=
  (ra₀.instances ra₀.anAct, (undeniable_root_forces_no_value
  ra₀).2.2.2, root_on_row_is_the_value)

```

```

/-- THE LOCK, one theorem on no axiom: the constructed root holds
with no hypothesis; every self-grounding
root holds by its deed; no given statement is the value on every
frame; the root read uniformly on the frames
gives no value; read on the row the root is the act, the act is the
value exactly, and both are keyed; the
closing theorem gives the value on every actual frame; no reading of
the kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1) ∧
  R ∧
  (∀ P : Prop, → ∀ F : Frame, (P → Value F)) ∧
  → (∀ F : Frame, R → Value F) ∧
  (∀ F : Frame, RootOnRow F → Realized F) ∧
  (∀ F : Frame, Realized F → Value F) ∧
  (RootOnRow calm ∧ → RootOnRow counter) ∧
  (Realized calm ∧ → Realized counter) ∧
  (∀ A : ActualEvens, Value A.F) ∧
  (→ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
  (∀ n : Nat, (∀ d, d < n → ∃ q, (staged n).realize d = some q) ∧ →
  Value (staged n)) :=
  (constructed_root_holds, G.instances G.anAct,
  given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
  root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
  act_is_keyed, goldbach_from_existence, record_wall,
  finite_record_never_forces)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the
closing theorem consumes existence read on
the row and nothing else; that reading is available on exactly the
frames where the value holds; and no
statement given without the reading is the value on every frame. One
theorem, on no axiom. -/
theorem from_existence_alone :
  (∀ A : ActualEvens, Value A.F) ∧
  (∀ F : Frame, Nonempty { A : ActualEvens // A.F = F } → Value F) ∧
  (∀ P : Prop, → ∀ F : Frame, (P → Value F)) :=
  (goldbach_from_existence, supply_iff, given_is_not_the_value)

end GBClose

/-! ## Cones, pinned as printed -/
/-- info: 'GBClose.sub_cancel_right' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.sub_cancel_right
/-- info: 'GBClose.sub_cancel_left' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.sub_cancel_left
/-- info: 'GBClose.sub_add_back' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.sub_add_back
/-- info: 'GBClose.primeAux_spec' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.primeAux_spec
/-- info: 'GBClose.rem_go_spec' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.rem_go_spec
/-- info: 'GBClose.rem_spec' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.rem_spec
/-- info: 'GBClose.below_next_multiple' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.below_next_multiple
/-- info: 'GBClose.rem_zero_gives_factor' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.rem_zero_gives_factor
/-- info: 'GBClose.factor_gives_rem_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.factor_gives_rem_zero
/-- info: 'GBClose.small_divisor' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.small_divisor
/-- info: 'GBClose.isPrime_sound' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.isPrime_sound
/-- info: 'GBClose.findAux_sound' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findAux_sound
/-- info: 'GBClose.findSplit_sound' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findSplit_sound
/-- info: 'GBClose.primeAux_false' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.primeAux_false
/-- info: 'GBClose.isPrime_complete' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.isPrime_complete
/-- info: 'GBClose.findAux_complete' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.findAux_complete
/-- info: 'GBClose.goldbach_gives_std' does not depend on any axioms
-/

```

```

#guard_msgs in #print axioms GBClose.goldbach_gives_std
/-- info: 'GBClose.std_gives_goldbach' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.std_gives_goldbach
/-- info: 'GBClose.goldbach_iff_std' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.goldbach_iff_std
/-- info: 'GBClose.mirror_involution' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.mirror_involution
/-- info: 'GBClose.split_mirrors' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.split_mirrors
/-- info: 'GBClose.seat_of_mirror' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.seat_of_mirror
/-- info: 'GBClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.calm_value
/-- info: 'GBClose.counter_fails' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.counter_fails
/-- info: 'GBClose.arith_value_iff' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arith_value_iff
/-- info: 'GBClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.root_given
/-- info: 'GBClose.root_conservative' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.root_conservative
/-- info: 'GBClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arrow_given
/-- info: 'GBClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.freedom_given
/-- info: 'GBClose.given_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.given_is_not_the_value
/-- info: 'GBClose.arrow_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.arrow_forces_nothing
/-- info: 'GBClose.given_in_both_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.given_in_both_worlds
/-- info: 'GBClose.act_is_the_value' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.act_is_the_value
/-- info: 'GBClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.act_is_keyed
/-- info: 'GBClose.act_is_goldbach' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.act_is_goldbach
/-- info: 'GBClose.act_is_goldbach_std' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.act_is_goldbach_std
/-- info: 'GBClose.goldbach_from_existence' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.goldbach_from_existence
/-- info: 'GBClose.goldbach_from_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.goldbach_from_the_act
/-- info: 'GBClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.supply_iff
/-- info: 'GBClose.every_even_lands' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.every_even_lands
/-- info: 'GBClose.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.gates_exclusive
/-- info: 'GBClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.nothing_escapes
/-- info: 'GBClose.counterexample_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.counterexample_refutes
/-- info: 'GBClose.check_means' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.check_means
/-- info: 'GBClose.executed_to_2000' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.executed_to_2000
/-- info: 'GBClose.below_height_decided' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.below_height_decided
/-- info: 'GBClose.certificate_does_not_reach_above' does not depend
on any axioms -/
#guard_msgs in #print axioms
GBClose.certificate_does_not_reach_above
/-- info: 'GBClose.certificate_is_load_bearing' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.certificate_is_load_bearing
/-- info: 'GBClose.weaker_do_not_give_binary' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.weaker_do_not_give_binary

```

```

/-- info: 'GBClose.row_split' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.row_split
/-- info: 'GBClose.remainder_not_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.remainder_not_forced
/-- info: 'GBClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.value_is_keyed
/-- info: 'GBClose.no_keyless_statement_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms GBClose.no_keyless_statement_is_the_act
/-- info: 'GBClose.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.pulse_does_not_certify
/-- info: 'GBClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.record_wall
/-- info: 'GBClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.fibre_is_two
/-- info: 'GBClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.prime_shape
/-- info: 'GBClose.additive_shape' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.additive_shape
/-- info: 'GBClose.two_axes_leave_two' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.two_axes_leave_two
/-- info: 'GBClose.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.three_axes_lock_one
/-- info: 'GBClose.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.finite_record_never_forces
/-- info: 'GBClose.uniform_step_forces_all' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.uniform_step_forces_all
/-- info: 'GBClose.goldbach_closure' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.goldbach_closure
/-- info: 'GBClose.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.denial_reenacts_root
/-- info: 'GBClose.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.external_proof_adds_nothing
/-- info: 'GBClose.undeniable_root_forces_no_value' does not depend
on any axioms -/
#guard_msgs in #print axioms GBClose.undeniable_root_forces_no_value
/-- info: 'GBClose.root_read_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.root_read_on_row_is_keyed
/-- info: 'GBClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.the_lock
/-- info: 'GBClose.from_existence_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.from_existence_alone
/-- info: 'GBClose.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.indicator_pos
/-- info: 'GBClose.root_satisfiable' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.root_satisfiable
/-- info: 'GBClose.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.constructed_root_holds
/-- info: 'GBClose.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.root_on_row_is_the_act
/-- info: 'GBClose.root_on_row_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.root_on_row_is_the_value
/-- info: 'GBClose.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.root_on_row_is_keyed
/-- info: 'GBClose.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.act_is_the_weakest_premise
/-- info: 'GBClose.existence_as_given_in_both_worlds' does not
depend on any axioms -/
#guard_msgs in #print axioms
GBClose.existence_as_given_in_both_worlds
/-- info: 'GBClose.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.nothing_given_refutes
/-- info: 'GBClose.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.only_refuter
/-- info: 'GBClose.findAux_least' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findAux_least

```



```

/-- info: 'GBClose.findSplit_least' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findSplit_least
/-- info: 'GBClose.even_double' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.even_double
/-- info: 'GBClose.split_iff_std' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.split_iff_std
/-- info: 'GBClose.seat_unique' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.seat_unique
/-- info: 'GBClose.mirror_has_one_seat' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.mirror_has_one_seat
/-- info: 'GBClose.uniform_premise_forces_nothing' does not depend
on any axioms -/
#guard_msgs in #print axioms GBClose.uniform_premise_forces_nothing
/-- info: 'GBClose.existence_as_given_forces_nothing' does not
depend on any axioms -/
#guard_msgs in #print axioms
GBClose.existence_as_given_forces_nothing
/-- info: 'GBClose.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.nothing_given_forces
/-- info: 'GBClose.counterexample_refutes_std' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.counterexample_refutes_std
/-- info: 'GBClose.executed_region' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.executed_region
/-- info: 'GBClose.executed_region_std' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.executed_region_std
/-- info: 'GBClose.arithUpTo_agrees' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.arithUpTo_agrees
/-- info: 'GBClose.arithUpTo_stops' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arithUpTo_stops
/-- info: 'GBClose.certificate_is_load_bearing_arith' does not
depend on any axioms -/
#guard_msgs in #print axioms
GBClose.certificate_is_load_bearing_arith
/-- info: 'GBClose.certificate_does_not_reach_above_arith' does not
depend on any axioms -/
#guard_msgs in #print axioms
GBClose.certificate_does_not_reach_above_arith
/-- info: 'GBClose.remainder_not_forced_arith' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.remainder_not_forced_arith
/-- info: 'GBClose.act_iff_above_2000' does not depend on any axioms
-/
#guard_msgs in #print axioms GBClose.act_iff_above_2000
/-- info: 'GBClose.act_iff_above_height' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.act_iff_above_height
/-- info: 'GBClose.shared_record_decides_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms GBClose.shared_record_decides_nothing
/-- info: 'GBClose.finite_arith_record_never_forces' does not depend
on any axioms -/
#guard_msgs in #print axioms
GBClose.finite_arith_record_never_forces
/-- info: 'GBClose.root_holds_on_every_frame' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.root_holds_on_every_frame
/-- info: 'GBClose.goldbach_from_root_on_row' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.goldbach_from_root_on_row
/-- info: 'GBClose.RA0_is_root_on_unit' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.RA0_is_root_on_unit
/-- info: 'GBClose.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms GBClose.the_lock_on_the_root

```

APPENDIX R • THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, TOE_Zero.lean, is carried verbatim from the root paper (Islam 2026h). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, im-

ports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x)$ (RA_0). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the_master_seal); it states the three counts side by side and no map between them. The row kernel repeats the definitions ΔE_0 , RA_0 and ra_0 verbatim; there the constructed root is the root on one existent ($RA_0_is_root_on_unit$), and it holds, forces no value, and the root read on the row of any frame is exactly that frame's value (the_lock_on_the_root). The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

```

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```

```

/-
TOE_Zero.lean · Theory of Theories of Everything (TOE of All
TOEs): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I · The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.
II · The constructed root. On the constructed domain the Root
Axiom is a theorem.
III · The Return and the line. On the integer quaternions i·j·k =
-1 and each unit squares to -1; conjugation is
an involution and its fixed set on the chart is exactly the
scalar line.
IV · The twelve gates, recovered three ways inside the integers of
the quaternions: the twenty-four Hurwitz units
counted twelve up to sign; the twelve even permutations with
class equation 1, 3, 4, 4; and the norm-two
shell of twenty-four, split twelve and twelve.
V · The regress terminated. Above a self-grounding root no level
grounds it further.

```

```

    VI · The master seal, every result bound in one theorem.
  -/
  namespace TOEO

  /-! ## I · The self-grounding root: the Omega Boundary -/

  /- A self-grounding root: acts occur, and every act instances the
    root. -/
  structure SelfGrounding (R : Prop) where
    Act      : Type
    anAct    : Act
    instances : Act → R

  /- A proposition whose denial hands it over. -/
  def SelfVerifying (P : Prop) : Prop := ¬P → P

  /- THE OMEGA BOUNDARY: every denial of the root is an act, and
    re-enacts it. -/
  theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
    (denial : G.Act) : R := G.instances denial

  /- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
    the act itself. -/
  theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
    G.instances G.anAct

  /- No external proof adds anything to the root. -/
  theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
    (Q : Prop) : (Q → R) ↔ R :=
    (fun _ => G.instances G.anAct, fun r _ => r)

  /- The deed of denying the root hands it over. -/
  theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
    SelfVerifying R := fun _ => G.instances G.anAct

  /- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
    root holds, every act re-enacts it, and no
    external proof adds to it. -/
  theorem the_floor_is_universal :
    ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
    Prop, (Q → R) ↔ R) :=
    fun _ G => (seated_undeniable G, denial_reenacts_root G,
    external_proof_adds_nothing G)

  /-! ## II · The constructed root -/

  /- The Root Axiom on the constructed one-point domain: its one
    existent actuates, at positive energy. -/
  def ΔE₀ : Unit → Int := fun _ => 1
  def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

  /- The constructed domain grounds its root: its one act is a deed,
    and the deed actuates. -/
  def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
    < 1 by decide)

  /- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
  theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
    ra₀

  /-! ## III · The Return and the line -/

  structure Q where
    (a b c d : Int)
    deriving DecidableEq

  def Q.mul (x y : Q) : Q :=
    (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
    y.a + x.c * y.d - x.d * y.c,
    x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
    y.c - x.c * y.b + x.d * y.a)

  def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
  def qone : Q := (1, 0, 0, 0)
  def qneg : Q := (-1, 0, 0, 0)
  def qi : Q := (0, 1, 0, 0)
  def qj : Q := (0, 0, 1, 0)
  def qk : Q := (0, 0, 0, 1)

  /- THE RETURN: i·j·k = -1. -/
  theorem the_return : (qi.mul qj).mul qk = qneg := by decide

  /- Each unit squares to -1. -/

```

```

theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

  /- The chart: every quaternion with coordinates in -2 ... 2. -/
  def chart : List Int := [-2, -1, 0, 1, 2]
  def quats : List Q :=
    chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

  set_option maxRecDepth 100000 in
  /- Conjugation is an involution on the chart. -/
  theorem conj_involutive_on_the_chart : quats.all (fun q =>
    q.conj.conj == q) = true := by decide

  set_option maxRecDepth 100000 in
  /- THE LINE: conjugation fixes exactly the scalar line on the
    chart. -/
  theorem the_line_is_the_fixed_set :
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true := by decide

  /-! ## IV · The twelve gates, recovered three ways -/

  /- Hurwitz quaternions in doubled coordinates: all even, or all
    odd. -/
  def hurwitz (a b c d : Int) : Bool :=
    (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
    0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
  /- Doubled coordinates -2 ... 2 hold every unit (doubled norm four)
    and every element of the norm-two shell (doubled
    norm eight), since a doubled coordinate of absolute value three
    or more already exceeds both norms. -/
  def doubled : List (Int × Int × Int × Int) :=
    chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

  /- The Hurwitz units: norm one, doubled norm four. -/
  def units : List (Int × Int × Int × Int) :=
    doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
    4 && hurwitz a b c d

  /- The first nonzero coordinate is positive: one representative of
    each pair ±u. -/
  def upToSign (p : Int × Int × Int × Int) : Bool :=
    let (a, b, c, d) := p
    if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
    0 < c else 0 < d

  set_option maxRecDepth 100000 in
  /- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
  theorem twenty_four_units_twelve_gates : units.length = 24 ∧
    (units.filter upToSign).length = 12 := by decide

  /- The norm-two shell: doubled norm eight. -/
  def shell : List (Int × Int × Int × Int) :=
    doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
    8 && hurwitz a b c d

  set_option maxRecDepth 100000 in
  /- THIRD WAY: the norm-two shell has twenty-four elements, split
    twelve and twelve by sign. -/
  theorem the_shell_splits_twelve_and_twelve :
    shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
    (shell.filter (fun p => !upToSign p)).length = 12 := by
    decide

  /- Permutations of four points, as their lists of images. -/
  def perms4 : List (List Nat) :=
    [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
    2, 3].flatMap fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
    == 4 then some [a, b, c, d] else none

  def inversions (p : List Nat) : Nat :=
    ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
    j)).countP
    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

  /- The twelve gates as the even permutations. -/
  def gates : List (List Nat) := perms4.filter fun p => inversions p %
    2 == 0

  def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0

```

```

def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
=> compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
| [] => [n]
| m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
| 0, _ => []
| _ + 1, [] => []
| fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
further ground of the root is idle; the root needs
no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
on the constructed domain; the Return closes;
the line is the fixed set; and the twelve gates are recovered
three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
: Prop, (Q → R) → R)) ∧
  RA₀ ∧ (q₁.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
= [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
  the_return, the_line_is_the_fixed_set,
  twenty_four_units_twelve_gates.2, the_class_equation.1,
  the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart

```

```

/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, *Triaxial_Actuation.lean* (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (*actuation_is_triaxial*, *registration_is_forced*, *the_only_special_cut*, *tongue_freedom*, *every_root_actuates*, *the_root_carries_three_axes*). The root's grade is read in the third kernel, R.3, *Root_Grade_Ledger.lean* (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026l), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (*a_warranted_root_is_theorem_grade*, *the_ledger_reads_the_warrant*, *the_grade_is_not_elected*, *ra_is_theorem_grade*, *coloc_grade_is_the_roots_grade*, *the_ledger_seal*). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026k), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
Triaxial_Actuation.lean · actuation is triaxial

```

Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every theorem rests on no

axiom at all, not even propositional extensionality, quotients or choice.

An actuation is a self-grounding root whose acts carry a direction: each act has a side and a height, and reversing the deed reverses the side and keeps the height. From the direction alone the kernel constructs three axes and proves them forced:

- I The fold: reversing the deed is an involution.
- II The registration: it keeps the height and forgets the side; it is blind to the fold, closed with it under combination, and it is the only height-keeping map onto the seat.
- III The seat: the fold's fixed set, where the fold and the registration both fix the act, and off which neither does.
- IV The third axis: off the seat the fold's orbit has two acts with one record, so the record leaves exactly the orientation open; every reading blind to the fold reads both alike.
- V The root through the axes: a reversed deed is still an act and instances the root; the axes are carried by every actuation, and they force no orientation.
- VI The bridge atom: the integers, with negation as the fold, are an actuation.
- VII The triaxial seal.
- VIII The Tongue: no reading of the record returns the orientation; the seat is the only special cut.
- IX Every root actuates: a self-grounding root is the root of an actuation over its own acts, so the root carries the three axes; the registration is the fold's record and lands on the seat, where the orientation an act and its reversal share stays open; the even pair leaves two acts, the orientation locks one; and a degenerate direction, one side only, is the characterless case.

-/

namespace TriAct

/-! ## The root, self-grounding -/

/- A self-grounding root: acts occur, and every act instances the root. -/

```
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R
```

/-! ## Actuation: a self-grounding root whose acts carry a direction -/

/- The direction of a deed: a side with a reversal, whose one fixed side is the zero side. -/

```
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero
```

/- An actuation: a self-grounding root whose acts are pairs of a side and a height. -/

```
structure Actuation (R : Prop) where
  dir      : Direction
  Height   : Type
  anAct    : dir.Side × Height
  instances : dir.Side × Height → R
```

variable {R : Prop}

/- Every actuation is a self-grounding root. -/

```
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)
```

/- The fold: reversing the deed reverses the side and keeps the height. -/

```
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
  A.Height := (A.dir.rev p.1, p.2)
```

/- The registration: it keeps the height and forgets the side. -/

```
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
  A.Height := (A.dir.zero, p.2)
```

/- The seat: the acts the fold fixes. -/

```
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
  fold A p = p
```

/-! ## I · the fold -/

```
theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
  A.Height) : fold A (fold A p) = p := by
  cases p with
  | mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
    [A.dir.rev_rev]
```

/-! ## II · the registration -/

/- The registration forgets the side: an act and its reversal have one record. -/

```
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
  A.Height) : reg A (fold A p) = reg A p := rfl
```

/- The registration keeps the height. -/

```
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
  A.Height) : (reg A p).2 = p.2 := rfl
```

/- The even pair is closed under combination: folding a record changes nothing. -/

```
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
  fold A (reg A p) = reg A p := by
  show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
    [A.dir.rev_zero]
```

/- The registration is idempotent. -/

```
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height) :
  reg A (reg A p) = reg A p := rfl
```

/- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the height and lands on the seat is

the registration. -/

```
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
  A.Height → A.dir.Side × A.Height)
  (keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
  g p = reg A p := by
  intro p
  have hs : (g p).1 = A.dir.zero := by
    have hf := lands p
    have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
    exact A.dir.fix_zero _ h1
  have hh := keeps p
  cases hg : g p with
  | mk s h =>
    rw [hg] at hs hh
    show (s, h) = (A.dir.zero, p.2)
    rw [show s = A.dir.zero from hs, show h = p.2 from hh]
```

/-! ## III · the seat -/

/- The seat is exactly the zero side. -/

```
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
  (fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
  fun h => by
    cases p with
    | mk s t =>
      show (A.dir.rev s, t) = (s, t)
      have hs : s = A.dir.zero := h
      rw [hs, A.dir.rev_zero])
```

/- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the registration both fix the act. -/

```
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : OnSeat A p) :
  fold A p = p ∧ reg A p = p := by
  refine (h, ?)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]
```

/- OFF THE SEAT BOTH STAND: neither the fold nor the registration fixes the act. -/

```
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
  (h : ¬ OnSeat A p) :
```

```

    fold A p ≠ p ∧ reg A p ≠ p := by
    refine (h, fun hr => h ?_)
    have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
    exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
  A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
  A.dir.Side × A.Height → α)
  (blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
  f (fold A p) = f p :=
  blind p

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
  Type} (k : A.dir.Side × A.Height → α)
  (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
  := rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
  A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.act

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
  × A.Height) (h : ¬ OnSeat A p) :
  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
  self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↦ (-d, t), the
registration (d, t) ↦ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
  fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧
  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
    = reg atom (d, t)) :=
  (seat_iff_zero_side atom (d, t),
  (fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
  fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
    ((seat_iff_zero_side atom (d, t)).mp hs)))

```

```

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
    : Int)) :=
  (fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
    (reg A p) = reg A p) ∧
  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
    (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p) → ∀ p, g p = reg
      A p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
    (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
      reg A p) :=
  (fold_involutive A, fun p => (reg_forgets_side A p,
    reg_keeps_height A p, fold_of_reg A p),
  registration_is_forced A, seat_cancels_both A, off_seat_both A,
  axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
      p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
    atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
      : Int))) :=
  (fun _ A => (fold_involutive A, seat_cancels_both A,
    fun p h => record_leaves_the_orientation A p h),
  fun d t => seat_iff_zero_side atom (d, t),
  the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (p : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : p (τ x) = p x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (p y) := by
  intro h
  have h1 : d (τ x) = g (p (τ x)) := congrFun h (τ x)
  have h2 : d x = g (p x) := congrFun h x
  rw [blind] at h1
  rw [h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :
  (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

```



```

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
    reversal alike and keeps the height, and two
    acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
  A.dir.Side × A.Height) :
  reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  (rfl, {fun h => show (reg A p).2 = (reg A q).2 from congrArg
    Prod.snd h, fun h => by
      cases p with
      | mk s t =>
        cases q with
        | mk s' t' =>
          have ht : t = t' := h
          show (A.dir.zero, t) = (A.dir.zero, t')
          rw [ht]})

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
    point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A (reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
    its reversal land on one seat point, the record
    of both; at that place the record has left exactly the orientation
    open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) :
  OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
    fold, the record among them, reads an act and
    its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) {α : Type}
  (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
  q) :
  fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
    the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
  A.dir.Side × A.Height)
  (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
    cases q with
    | mk s' t' =>
      have ht : t = t' := congrArg Prod.snd hr
      have hs' : s = s' := hs
      rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
    on the seat, and the fold and the
    registration are both the identity: one scalar, no fold, no seat to
    lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
  A.dir.Degenerate) (p : A.dir.Side × A.Height) :
  OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
    actuation over its own acts, with the integer
    direction; the root's deeds sit at the zero side and instance the
    root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
  (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  {fun _ => rfl,

```

```

  fun a h =>
    have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
      ((1 : Int), a)).mpr h
    absurd h0 (by decide),
    intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
    actuation carries the fold, an involution; the
    registration, keeping the height, forgetting the side, the only
    height-keeping map onto the seat; and the seat,
    where both fix the act; and it is not characterless: at every height
    an act off the seat, its reversal a second
    act with the same record, both instantiating the root, the orientation
    between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
    S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
    OnSeat S.toActuation (g p)) →
    ∀ p, g p = reg S.toActuation p) ∧
  (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
    S.toActuation p = p) ∧
  (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
    reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
    S.toActuation ((1 : Int), a)) ∧
  (∀ p, ¬ OnSeat S.toActuation p →
    R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
    S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
  fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
    S.toActuation p),
  registration_is_forced S.toActuation,
  seat_cancels_both S.toActuation,
  fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
    a) ((every_root_actuates S).2.1 a),
  axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
    axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
    -/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
    -/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
    any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
    axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
    on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
    axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
    axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
  Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
  co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no
axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I   The grade ledger and the weakest-link join, as in the Codex.
II  The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV  The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V   Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI  The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its
statement, is theorem grade; a root posited without one, the
Codex's `axiom RA`, is premise grade;
the two claims of section IV carry exactly the grades this
rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

```

```

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
|.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
  stmt    : Prop
  grade   : Grade
  warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
  stmt    := c₁.stmt ∧ c₂.stmt
  grade   := Grade.weakest c₁.grade c₂.grade
  warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
  Grade.weakest c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>

```

```

    (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i
    % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero
  target_odd_at_zero
end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim₀ : Claim where
  stmt    := RA₀
  grade   := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt    := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade   := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
the same statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt    := RA₀
  grade   := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
= g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
theorem, so the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
  raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
hypothesis; the wall is a theorem; their
co-location holds and stands at theorem grade; and co-location's
grade is the root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA₀ ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim₀.stmt ∧ colocClaim₀.grade = .theorem ∧

```

```

  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it
-/

/-- The evidence a ledger entry carries for a statement: posited
with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
is a proof with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem
grade; a root posited with none, as the Codex
declares 'axiom RA', is premise grade. -/
def rootGrade : Evidence RA₀ → Grade
| .warranted _ => .theorem
| .posited      => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
:= rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
reads premise, the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
.premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
carries the grade the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none.
-/
theorem the_ledger_reads_the_warrant :
  raClaim₀.grade = rootGrade (.warranted raClaim₀.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
and premise from none, never the other way;
whatever label an entry carries, the warranted root's grade under
the rule is theorem. -/
theorem the_grade_is_not_elected :
  (∀ w : RA₀, rootGrade (.warranted w) = .theorem) ∧ rootGrade
  .posited ≠ .theorem ∧
  (∀ w : RA₀, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
  Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-! info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-! info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-! info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-! info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-! info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-! info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-! info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-! info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-! info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-! info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/

```

```

#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below (Islam 2026j).

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the kernel is called axiom-free throughout only because every one of its theorems prints no axiom, those that pass through the arithmetic bridge included, once the remainder was read from the definition of % itself.

The register. In the programme's vocabulary: the form, the act's identity with the value and the statement, the universal closure, the executed region and the typed citations stand at $[\{\epsilon\}, T]$, one hundred four theorems on no axiom; the triaxial reading at $[\{\epsilon\}, S]$; the value above the certified height at premise grade on the act; $\Delta M = 0$ on the cited number theory. The act is the row's least-erasure posit read as existence realized; the row carries no defeater. The freedom cut, prime-as-freedom in its multiplicative and additive faces, and the triaxial lock are applied on the row, with the lessons of the Riemann, P versus NP, Navier--Stokes, Hodge and Yang--Mills closures (Islam 2026e, 2026d, 2026a, 2026b, 2026i) combined, and the seat of the mirror read as the programme's seat.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (*every_root_actuates, degenerate_is_characterless*). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat the registration is the only height-keeping map onto the seat the axes force no orientation	every_root_actuates, the_root_carries_three_axes, actuation_is_triaxial, registration_is_forced (Appendix R.2) registration_is_forced (R.2) axes_force_no_orientation (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	fold, denial_instances_root (R.2); denial_reenacts_root, denial_instantiates (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	registration_is_the_folds_record, reg_forgets_side, reg_keeps_height, fold_involution, fold_of_reg (R.2) readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026f, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)
the bridge atom is an actuation, and not characterless	the_bridge_atom, the_atom_is_not_characterless, the_triaxial_seal (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)

Clause of the statement	Receipt
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement
the root alone forces no value; the root read on the row is the value	undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root ra_0 of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; the row's own seat is the mirror's one fixed point on $[0, n]$ (mirror_involution, mirror_has_one_seat), a statement of this kernel beside the actuation's seat; the row ledger of Islam (2026f, Book III), cited

REFERENCES

- Chen, J. R. 1973. On the representation of a larger even integer as the sum of a prime and the product of at most two primes. *Scientia Sinica* 16: 157--176.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE* 28, 625--635.
- Goldbach, C. 1742. Letter to L. Euler, 7 June 1742. In P.-H. Fuss (ed.), *Correspondance mathématique et physique de quelques célèbres géomètres du XVIIIème siècle*, vol. 1. St. Petersburg, 1843.
- Hardy, G. H. and J. E. Littlewood. 1923. Some problems of 'Partitio Numerorum'; III: On the expression of a number as a sum of primes. *Acta Mathematica* 44: 1--70.
- Helfgott, H. A. 2013. The ternary Goldbach conjecture is true. arXiv:1312.7748.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. What exists is realized: the completed formal closure of the Hodge question. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026c. The cut-agnostic division theorem: why every open problem is exactly its proved part and its unicorn. Zenodo. doi:10.5281/zenodo.22954862.

- Islam, M. F. 2026d. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026e. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026g. PhysOS^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026h. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026i. The floor under every confined field: a formal completed closure of Yang–Mills existence and the mass gap from existence alone. Zenodo. doi:10.5281/zenodo.23162231.
- Islam, M. F. 2026j. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Islam, M. F. 2026k. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026l. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapient/Trisduction, GitHub.
- Oliveira e Silva, T., S. Herzog and S. Pardi. 2014. Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$. *Mathematics of Computation* 83: 2033–2060.

End of manuscript